ECE 2774 - Integrating PV Generation

Bailey Stout

5/2/2025

Contents

1 Project Overview

Introduction and Overview

This document details a Python-based simulation tool designed for the analysis of electric power systems. The tool provides a modular, object-oriented framework for building power network models and performing fundamental analyses, including power flow calculations and short-circuit fault studies. Users can define system components such as buses, generators, loads, transmission lines, and transformers, assemble them into a circuit, and then utilize various algorithms to simulate the system's behavior under different operating conditions. The primary goal is to offer a flexible platform for use in any professional context.

Circuit Architecture

The core of the simulation architecture is the `Circuit` class, which acts as the central container and manager for all power system components and simulation settings. The system topology is built by adding instances of various component classes to the `Circuit` object. For example, some of the component classes used to create a circuit object are:

- `Bus`: Represents connection points (nodes) in the network.
- `Load`: Models power consumption at a bus.
- `Generator`: Represents power sources connected to buses, including the designation of a slack bus and PV buses.
- `TransmissionLine`: Models the lines connecting buses, calculable either from detailed physical parameters (using `Conductor`, `Bundle`, `Geometry` helper classes) or directly from per-unit R, X, and B values.
- `Transformer`: Models transformers connecting buses at different voltage levels, including various winding configurations (Y-Y, Y-D, D-Y, D-D) and grounding options.

Global parameters like system frequency and power base are managed via the `Settings` class. The resistive characteristics of the network are represented by the system admittance matrix (`Ybus`), which is assembled by the `Circuit` class based on the added components. For fault studies, sequence admittance matrices (`Y0bus`, `Ypbus`, `Ynbus`) are also constructed.

Power Flow Solution

The tool provides several methods for analyzing the steady-state operation of the modeled power system, primarily through power flow calculations. These methods determine the voltage magnitude and phase angle at each bus, as well as the real and reactive power flowing through the network components. The implemented power flow algorithms are within the `Solution.py` module and include:

- **Newton-Raphson Method** (`NewtonRaphson` class): An iterative, full AC power flow method with user-desired accuracy. It solves the non-linear power balance equations using Jacobian matrix updates. It includes logic for handling generator reactive power limits.
- **Fast Decoupled Load Flow** (`FastDecoupled` class): An approximation of the Newton-Raphson method that is more efficient for use in some contexts. This algorithm returns a similar level of accuracy as the Newton-Raphson method.

- **DC Power Flow** (`DCPowerFlow` class): A linearized approximation that neglects reactive power and assumes flat voltage magnitudes (1 p.u.). It solves a linear system involving only bus angles and real power injections, providing a quick estimate of real power flows and angles.

These solution methods are invoked via corresponding methods within the `Circuit` class (e.g., `do_newton_raph()`) and update the state variables (voltages, angles) stored within the `Bus` objects and power outputs in the `Generator` objects.

Fault Simulation

Beyond steady-state analysis, the tool is equipped to simulate the impact of short-circuit faults on the power system. This capability is crucial for designing protection systems and assessing system security. Both balanced (symmetrical) and unbalanced (unsymmetrical) faults can be analyzed:

- **Symmetrical Faults** (`ThreePhaseFault` class): Simulates a balanced three-phase fault at a specified bus. This analysis utilizes the positive-sequence impedance network only, simplifying the calculation. The class computes the total fault current and the post-fault voltage profile across all system buses using the faulted bus impedance matrix derived from generator subtransient reactances.
- **Unsymmetrical Faults** (`UnsymmetricalFaults` class): Simulates unbalanced faults, specifically single line-to-ground (SLG), line-to-line (LL), and double line-to-ground (DLG) faults. This analysis relies on the method of symmetrical components, requiring the construction of zero-, positive-, and negative-sequence impedance networks (`Z0bus`, `Zpbus`, `Znbus`). The class calculates sequence currents and voltages for the specific fault type and converts them back to phase quantities to determine the three-phase fault currents at the fault location and the post-fault three-phase voltage profile across the system. Fault impedance (Z_f) can be included in the calculations.

Both fault simulation classes utilize some dedicated parameter calculation classes including (`ThreePhaseFaultParameters`, `UnsymmetricalFaultParameters`) to perform the core fault calculations.

Summary

This Python power system simulation tool provides a comprehensive framework for modeling and analyzing power systems. It allows users to construct circuit models by defining buses, generators, loads, lines, and transformers. The tool supports standard steady-state analysis through multiple power flow algorithms (Newton-Raphson, Fast Decoupled, DC Power Flow) and enables transient analysis via short-circuit fault simulations for both balanced (three-phase) and unbalanced (SLG, LL, DLG) conditions using symmetrical components. With its modular design and range of analytical capabilities, the tool serves as a valuable asset for learning, teaching, and performing foundational research in power system engineering.

2 Class Diagram and Documentation

Any variable with a red square next to it must be passed to the class `__init__` function, and any variable with a non-filled circle next to it is a class variable. Class methods are described with filled green circles.

2.1 Bundle

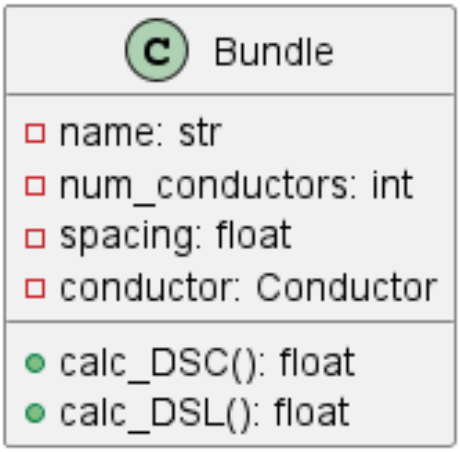


Figure 1: Bundle Class

The `Bundle` class models a bundled conductor configuration for transmission lines; it represents how many conductors are present for each phase. The class initializes with a bundle name, number of conductors, conductor spacing, and a `Conductor` object. The class computes the bundle's Geometric Mean Distance (DSC) and Geometric Mean Radius (DSL) using `calc_DSC()` and `calc_DSL()`, respectively. These calculations depend on the conductor's radius, geometric mean radius (GMR), and spacing configuration, following standard formulas for different bundle configurations. The computed DSC and DSL values are essential for analyzing transmission line parameters.

2.2 Bus

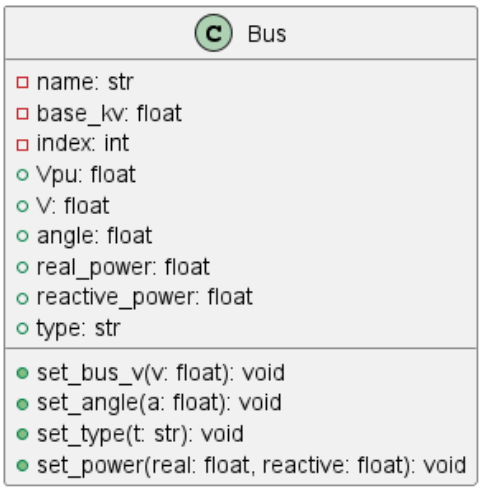


Figure 2: Bus Class

The `Bus` class represents a bus in the power system, serving as a node where power injections, loads, and interactions occur. Each bus is defined by a name, base voltage level (in kV), and an index for system identification, which is handled internally. The class initializes with default values for voltage magnitude (`v` = 1 p.u.), phase angle (`angle` = 0.0 degrees), real power injection (`real_power` = 0.0 MW), and reactive power injection (`reactive_power` = 0.0 MVAR). Bus objects have three types: PQ (load bus), PV (generator bus), and Slack (reference bus). Only one bus can be the slack bus, and it must have a generator connected to it. Bus objects are initialized to PQ by default, and their type is modified as needed internally.

Note 1: If two buses with the same name are created, the first bus will be accepted, but the second bus will be ignored, and the software will display a warning. Note 2:

Bus objects need to be created before any component objects. Connecting a component to a bus that does not exist will display an error message.

2.3 Circuit

The Circuit.py module is used to operate the `Circuit`, `ThreePhaseFault`, and `UnsymmetricalFaults` classes.

2.3.1 Circuit Class

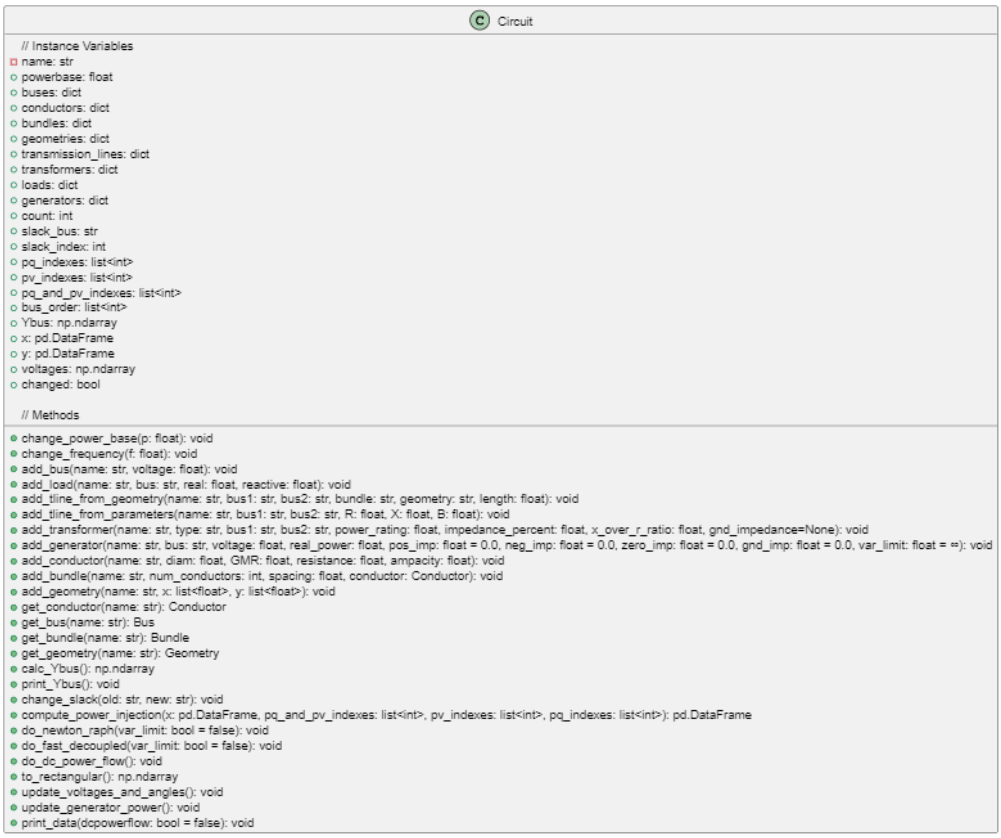


Figure 3: Circuit Class

The `Circuit` class serves as the central organizational structure for simulating a power system network. It stores and manages components such as buses, transmission lines, transformers, loads, generators, and various system parameters. Through this class, one can define a complete circuit topology, modify its base settings, compute its admittance matrix, and prepare it for power flow analysis.

Attributes

- `name`: A string that denotes the name of the power circuit. This acts as a simple identifier for data handling.
- `powerbase`: The system-wide power base, imported from a global `settings` module. This is used for per-unit conversions.
- `buses, conductors, bundles, geometries, transmission_lines, transformers, loads, generators`: These are dictionaries that store the corresponding system

elements. The keys are string identifiers (names), and the values are instances of each respective class (e.g., `Bus`, `Load`, `Transformer`, etc.).

- `count`: An internal counter used to assign incremental indices to each new bus.
- `slack_bus`: The name assigned to the slack bus object.
- `slack_index`: The numerical index of the slack bus in the system matrix.
- `pq_indexes, pv_indexes, pq_and_pv_indexes`: Lists of bus indices used for power flow computations. PQ buses are those with specified real and reactive power; PV buses are those with specified voltage magnitude and real power.
- `bus_order`: Maintains the order in which buses are added for display and indexing purposes.
- `Ybus`: The admittance matrix of the system, computed via the `calc_Ybus()` method. It is an $n \times n$ complex matrix.
- `x, y`: Placeholder attributes that store the results of power flow computations. Typically, `x` represents bus voltage magnitudes and phase angles, while `y` stores net power injections.
- `voltages`: Voltages stored from algorithm results.
- `changed`: Boolean indicator to show if there have been any components changed. This is used to ensure power flow algorithms execute smoothly.

Constructor

```
def __init__(self, name: str)
```

Initializes an empty circuit object with a name and sets up all the internal dictionaries and tracking variables. Also initializes the power base from the global settings.

Methods

- **change_power_base(p: float)**
  Updates the power base of the system to a new value `p` in MVA and modifies the global settings accordingly.
- **change_frequency(f: float)**
  Changes the global frequency setting of the power system via the `settings` module.
- **add_bus(name: str, voltage: float)**
  Adds a bus with the specified name and base voltage to the circuit. Each bus is assigned a unique index. Initially, all buses are assumed to be PQ buses.
- **add_load(name: str, bus: str, real: float, reactive: float)**
  Adds a load to the specified bus, with the given real and reactive power values in MW and MVar, respectively. The corresponding bus is updated with the load's negative power injection.
- **add_tline_from_geometry(name: str, bus1: Bus, bus2: Bus, bundle: Bundle, geometry: Geometry, length: float)**
  Adds a transmission line between two buses, using data from a desired `Bundle` and `Geometry`. The transmission line is stored in the internal dictionary.

- **add_tline_from_parameters(name: str, bus1: Bus, bus2: Bus, R: float, X: float, B: float)**
  Adds a transmission line directly from per-unit impedance parameters: resistance `R`, reactance `X`, and shunt admittance `B`.

- **add_transformer(name: str, type: str, bus1: Bus, bus2: Bus, power_rating: float, impedance_percent: float, x_over_r_ratio: float, gnd_impedance=None)**
  Adds a transformer between two buses with nameplate data. Ground impedance is optional. The transformer's admittance contribution is included during `Ybus` calculation.

- **add_generator(name: str, bus: str, voltage: float, real_power: float, pos_imp = 0.0, neg_imp = 0.0, zero_imp = 0.0, gnd_imp = 0.0, var_limit = float('inf'))**
  Adds a generator to a specified bus. The first generator automatically designates the associated bus as the slack bus. Additional generators are treated as PV buses. Generator properties such as sequence impedances and reactive power limits can be specified.

- **add_conductor(name: str, diam: float, GMR: float, resistance: float, ampacity: float)**
  Adds a conductor type with specified properties for future use in bundle definitions.

- **add_bundle(name: str, num_conductors: int, spacing: float, conductor: Conductor, v=765.e3)**
  Defines a bundled conductor configuration for use in geometric transmission line modeling.

- **add_geometry(name: str, x: list[float], y: list[float])**
  Defines the spatial phase arrangement for transmission line modeling using coordinate arrays `x` and `y`.

- **get_conductor(name: str)**
  Returns a conductor with given name.

- **get_bus(name: str)**
  Returns a bus with given name.

- **get_bundle(name: str)**
  Returns a bundle with given name.

- **get_geometry(name: str)**
  Returns geometry with given name.

- **calc_Ybus()**
  Constructs the admittance matrix by aggregating primitive admittance matrices from all transmission lines and transformers. The matrix is stored internally as a complex NumPy array and returned for use in power flow solvers.

- **print_Ybus()**
  Prints a rounded, full-resolution version of the admittance matrix using pandas DataFrame for improved readability. Column and row labels are based on bus indices.

- **change_slack(old: str, new: str)**
  Reassigns the slack bus from one node to another, provided the new node is a PV bus. The old slack bus is changed to a PV bus.

- **compute_power_injection(x, pv_indexes=None, pq_indexes=None)**
  Computes the active (P) and reactive (Q) power injections at each bus, using the current state variable vector x. Optionally accepts PV and PQ bus indices to accommodate for internal changes to buses. The function returns a vector of power mismatches labeled by bus, combining P_k and Q_k values.

- **do_newton_raph()**
  Executes the Newton-Raphson power flow solution using the current state of the network. Updates bus voltages, angles, and generator power outputs based on the converged solution.

- **do_fast_decoupled()**
  Executes the Fast Decoupled Load Flow solution using the current circuit configuration. After solving, the voltage magnitudes, angles, and generator outputs are updated.

- **do_dc_power_flow()**
  Solves the DC Power Flow approximation for the network. Updates bus voltage angles and generator powers accordingly.

- **to_rectangular()**
  Converts the magnitude and angle values of the bus voltages into rectangular complex voltages.

- **update_voltages_and_angles()**
  Synchronizes the voltage magnitudes and phase angles stored in each bus object based on the most recent power flow solution vector x.

- **update_generator_power()**
  Updates the real and reactive power values of all generators based on the most recent power flow results. Converts per-unit quantities to MW and MVAR using the system power base.

- **print_data(dcpowerflow=False)**
  Prints a formatted table of bus data, including bus voltages, angles, loads, and generation values. If `dcpowerflow=True`, reactive power information is excluded from the table.

Overall, the `Circuit` class abstracts the topology and physical/electrical parameters of a power system into an object-oriented framework. It serves as the primary container for both data setup and analytical procedures like power flow computation.

2.3.2 ThreePhaseFault Class

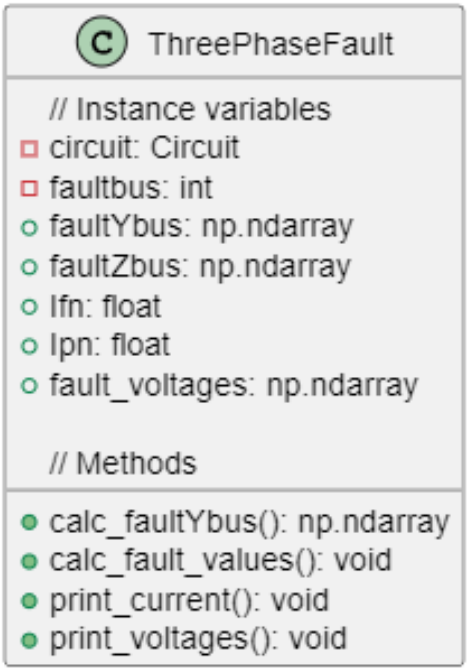


Figure 4: ThreePhaseFault Class

The `ThreePhaseFault` class models the behavior of a power system during a balanced three-phase fault condition at a specified bus. It is initialized with a `Circuit` object, the bus number at which the fault occurs, and the post-fault voltage magnitude. Upon initialization, it computes the modified bus admittance matrix (`faultYbus`) by adjusting the diagonal elements of the original `Ybus` matrix to include the inverse of each generator's sub-transient reactance. The inverse of this matrix, `faultZbus`, is also computed to facilitate fault current and voltage calculations.

The `calc_fault_values()` method computes the fault current and resulting bus voltages using the `ThreePhaseFaultParameters` solver class, storing the computed current (`Ifn`) and voltages (`fault_voltages`) as class attributes. The `print_current()` method prints the magnitude and angles of the fault current for phases A, B, and C. Similarly, the `print_voltages()` method provides a display of the post-fault voltage magnitudes and angles for each bus and each phase, synthesized from the positive sequence solution. This class is useful for short-circuit and protection analysis.

2.3.3 UnsymmetricalFaults Class

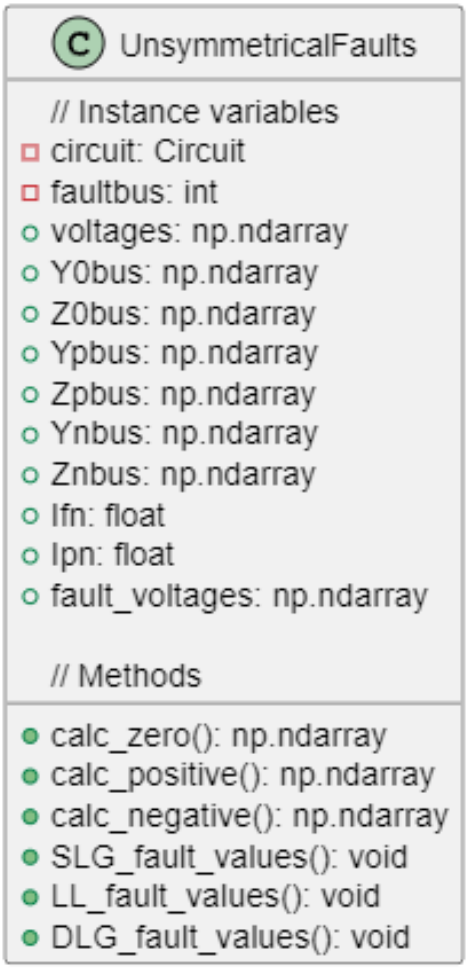


Figure 5: UnsymmetricalFaults Class

The `UnsymmetricalFaults` class models unbalanced faults—specifically single line-to-ground (SLG), line-to-line (LL), and double line-to-ground (DLG) faults—within a three-phase power system. It is initialized with a `Circuit` object, the faulted bus index, and a post-fault voltage magnitude. Upon creation, the class computes the sequence admittance matrices for zero (`Y0bus`), positive (`Ypbus`), and negative (`Ynbus`) sequences by incorporating the appropriate sequence impedances of generators, transformers, transmission lines, and loads. Each admittance matrix is then inverted to obtain the corresponding impedance matrices `Z0bus`, `Zpbus`, and `Znbus`.

Sequence admittances are calculated in dedicated methods. The `calc_zero()` method assembles the zero-sequence network by summing the zero-sequence primitive admittances of system elements. `calc_positive()` builds the positive-sequence network similarly but also includes load admittances derived from complex power and bus voltage. `calc_negative()` mirrors this process using the negative-sequence impedance for each generator and includes load admittances accordingly. Once these matrices are established, various fault calculation methods—`SLG_fault_values()`, `LL_fault_values()`,

and `DLG_fault_values()`—are available to compute sequence currents and faulted bus voltages. These methods utilize the `UnsymmetricalFaultParameters` solver from the `Solution` module.

After solving, the class stores the resulting fault current (`Ifn`), phase currents (`Ipn`), and voltage profile (`fault_voltages`). The user can visualize the results using `print_current()`, which displays the phase current magnitudes and angles, and `print_voltages()`, which tabulates the bus voltages in all three phases along with their angles. The class also includes utility methods—`print_Y0bus()`, `print_Ypbus()`, and `print_Ynbus()`—to display the admittance matrices for inspection. Overall, this class enables analysis of unsymmetrical faults using a sequence network framework.

2.4 Component

The Component.py module is used to operate the `Load` and `Generator` classes.

2.4.1 Load

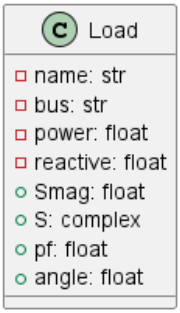


Figure 6: Load Class

This module defines electrical components used in circuit simulations, with `Load` and `Generator` as key classes. The `Load` class represents a power system load at a specified bus, characterized by real and reactive power consumption. It calculates the power factor angle using `calc_angle()`. The `Generator` class models a power system generator at a specific bus, defined by its voltage, real power output, and positive-sequence impedance. These component classes provide the foundation for constructing and analyzing electrical circuits within the power system.

2.4.2 Generator

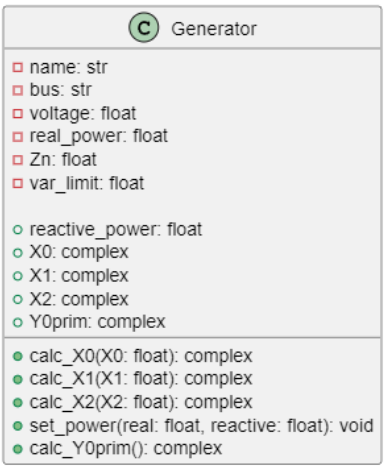


Figure 7: Generator Class

The `Generator` class represents a generator connected to a specific bus in the power system. It is characterized by its terminal voltage, real power output, and sequence impedances for fault analysis. The generator's subtransient (positive-sequence), negative-sequence, and zero-sequence reactances are internally converted to per-unit values on

the system power base. The `Y0prim` admittance is calculated from the zero-sequence impedance and grounding impedance for use in building the zero-sequence bus admittance matrix (`Y0bus`). Additionally, the generator supports dynamic adjustment of real and reactive power through the `set_power()` method. The optional `var_limit` parameter can be used to constrain the reactive power generation during power flow calculation.

2.5 Conductor

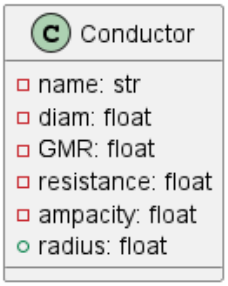


Figure 8: Conductor Class

The `Conductor` class models a transmission line conductor. Each conductor is defined by its name, outside diameter (in inches), geometric mean radius (GMR, in feet), resistance at 50°C and 60 Hz, and rated ampacity. The constructor calculates the conductor's radius in feet by converting from inches. These attributes are crucial for determining line impedance, calculating inductance and capacitance, and performing transmission line parameter analysis. This class provides a structured way to define conductors for use in bundled transmission lines and broader power system simulations.

2.6 Constants

The `Constants` module defines fundamental constants used in power system calculations. These include the imaginary unit (`j`), the permittivity of free space (`epsilon` = 8.854×10^{-12} F/m), and unit conversion factors for length: inches to meters (`inch2m` = 0.0254), feet to meters (`feet2m` = 0.3048), and miles to meters (`mi2m` = 1609.34). These constants are useful for various computations within the system.

2.7 Geometry

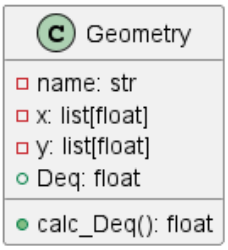


Figure 9: Geometry Class

The `Geometry` class represents the physical arrangement of conductors in a transmission line, defining their spatial coordinates and calculating the equivalent geometric mean distance (Deq). The class is initialized with a name and lists of x and y coordinates representing conductor positions. The `calc_Deq` method computes Deq using the geometric mean of conductor spacings, essential for determining transmission line parameters such as inductance and capacitance. This class provides a structured approach to modeling conductor layouts for power system analysis.

2.8 Settings

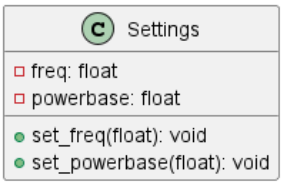


Figure 10: Settings Class

The `Settings` class defines global system parameters for power system calculations, specifically the system frequency and base power. These values are fundamental in per-unit system analysis and power flow studies. The class initializes with default values of 100 MVA for `powerbase` and 60 Hz for `freq`, but these can be adjusted using the `set_powerbase` and `set_freq` methods. This provides flexibility in adapting the simulation to different power system conditions while maintaining consistency across calculations.

2.9 Solution

The Solution.py module is used to operate the `NewtonRaphson`, `FastDecoupled`, `DCPowerFlow`, and `ThreePhaseFaultParameters` classes.

2.9.1 NewtonRaphson Class

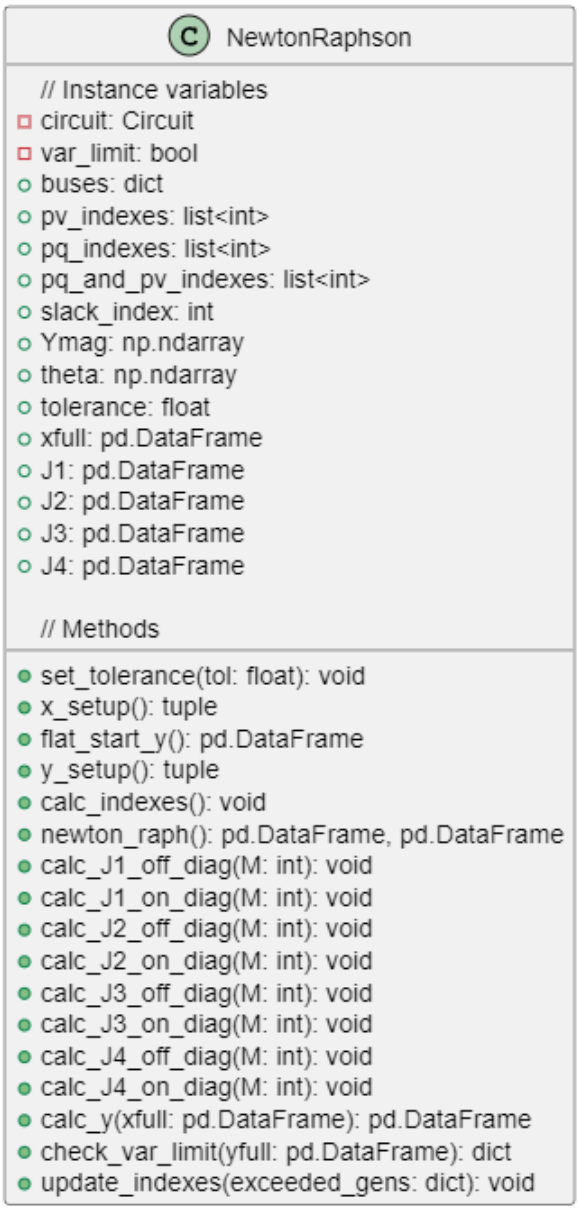


Figure 11: NewtonRaphson Class

The `NewtonRaphson` class is designed to perform power flow analysis using the Newton-Raphson method. It interfaces with a `Circuit` object to access bus and admittance data, classify bus types, and iteratively solve the system governing active and reactive

power balance at each bus. The method offers precision at a desired tolerance and convergence once solution conditions are satisfied.

Attributes

- `circuit`: Reference to a `Circuit` object, which contains system data such as the bus admittance matrix, generator/load specifications, and bus classifications.
- `buses`: Copy of circuit object's buses to be used for handling any bus changes.
- `pv_indexes`, `pq_indexes`, `pq_and_pv_indexes`: Lists of bus indices for PV and PQ buses. This initially matches the indexes of the circuit and changes dynamically if a VAR limit is reached.
- `slack_index`: Index of the slack (reference) bus.
- `Ymag`, `theta`: Magnitude and angle (in radians) of the complex admittance matrix Y_{bus}.
- `tolerance`: Convergence threshold for the maximum power mismatch.
- `xfull`: Full state vector $[\delta_1, \delta_2, \ldots, \delta_n, V_1, V_2, \ldots, V_n]^T$.
- `J1, J2, J3, J4`: Submatrices of the Jacobian matrix used in Newton-Raphson iterations.
- `var_limit`: Boolean used to determine if VAR limiting will occur in algorithm.

Constructor

```
def __init__(self, circuit: Circuit)
```

Initializes a solution object with a circuit passed as a parameter. By default the tolerance is set to 0.001.

Methods

- **set_tolerance(tol: float)**
  Sets the convergence tolerance for iterations.
- **x_setup()**
  Initializes the voltage magnitude and angle vector (x) for all buses, excluding slack and fixed voltage PV buses.
- **flat_start_y()**
  Set y vector to flat start values to be used for computing power injection.
- **y_setup()**
  Generates the expected power injection vector y.
- **calc_indexes()**
  Establish indexes to be used in power injection. Specifically, this concatenates current pq and pv bus indexes.
- **newton_raph()**
  Executes the Newton-Raphson power flow algorithm. Returns the converged voltage state and power injection vector.

- **calc_J1_off_diag, calc_J1_on_diag, calc_J2_off_diag, calc_J2_on_diag, calc_J3_off_diag, calc_J3_on_diag, calc_J4_off_diag, calc_J4_on_diag**
  The calc_J functions are all passed with the total number of buses in the circuit minus one. Each of these functions calculate a portion of the Jacobian matrix according to the following formula:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial P}{\partial \delta} & \frac{\partial P}{\partial V} \\ \frac{\partial Q}{\partial \delta} & \frac{\partial Q}{\partial V} \end{bmatrix} = \begin{bmatrix} J_1 & J_2 \\ J_3 & J_4 \end{bmatrix}$$

- **calc_y(xfull: pd.DataFrame)**
  Computes the real and reactive power injections at each bus using the full system state vector, which contains voltage angles and magnitudes. This function returns a DataFrame `y` with power injection values. Each row is labeled `P1, P2, ..., PN` for real power and `Q1, Q2, ..., QN` for reactive power.

- **check_var_limit(yfull: list)**
  Check to see if any VAR limits have been exceeded in generators based on y vector.

- **update_indexes(exceeded_gens: dict)**
  Based on any VAR limits being exceeded, update indexes such that NewtonRaphson can be calculated again with new VAR limit constraint.

2.9.2 FastDecoupled

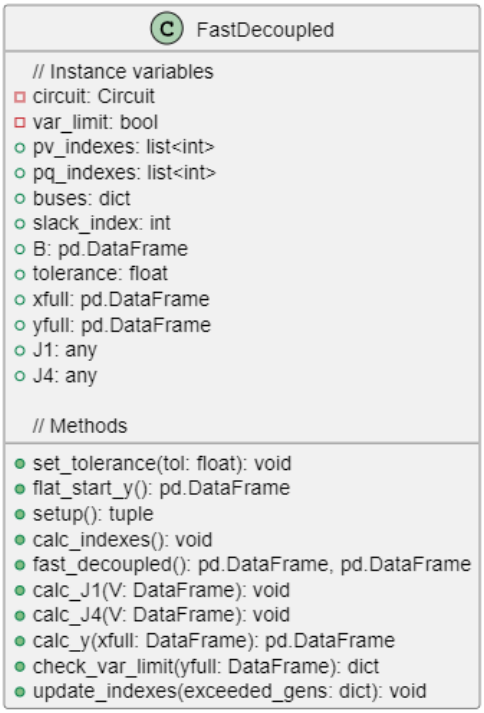


Figure 12: FastDecoupled Class

The `FastDecoupled` class implements the fast decoupled power flow algorithm, a different method for solving the power flow problem in a given power system. The class operates on a user-defined `Circuit` object that provides network topology, bus data, admittance matrix, and element indices.

The class initializes with the circuit's admittance matrix, isolates its imaginary part for use in decoupling, and stores other necessary data structures such as the Jacobian submatrices and convergence tolerance. The `setup()` method builds the initial x and y vectors in preparation for iterative solving.

The `fast_decoupled()` method is the core routine. It iteratively updates voltage angles and magnitudes using separate Jacobian submatrices, `J1` and `J4`, which are computed with the `calc_J1()` and `calc_J4()` methods, respectively. These matrices are formed using reduced versions of the susceptance matrix and voltage magnitude vectors. The algorithm continues to iterate until the power mismatch vector falls below the convergence threshold, at which point the solution vectors for voltage and angle are returned alongside the calculated bus power injections.

The `calc_y()` method computes the complex power injection vector y (comprising active P and reactive Q power) based on the current state vector x_{full}, which contains voltage magnitudes and angles at each bus.

2.9.3 DCPowerFlow

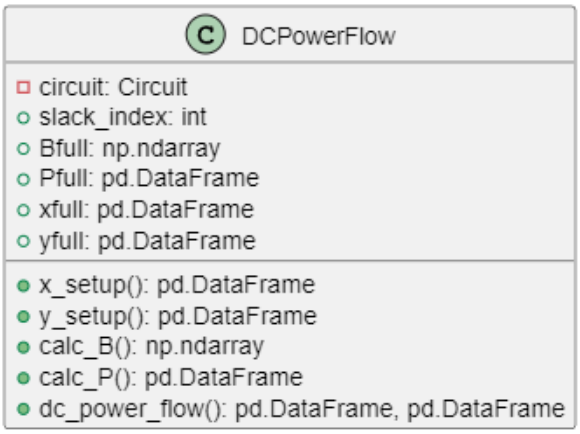


Figure 13: DCPowerFlow Class

The `DCPowerFlow` class implements the DC power flow algorithm. This method assumes flat voltage magnitudes and neglects reactive power, focusing solely on active power flows. Upon initialization, the class receives a `Circuit` object containing the system admittance matrix, bus data, and system base power. It precomputes the susceptance matrix B from the imaginary part of the bus admittance matrix, the active power injection vector P, the full state vector x_{full}, and the expected output vector y_{full}. The state vector includes bus voltage angles δ and magnitudes V, while the output vector contains active and reactive power injections, although only active power is relevant in this method.

The `dc_power_flow()` method solves the linear system $B'\delta = -P$, where B' is the reduced susceptance matrix with the slack bus removed, and δ is the vector of bus voltage angles. After solving for the angles, it updates the corresponding entries in the full state vector. The method also calculates the power injection at the slack bus based on the system configuration and updates the power vector accordingly. This is done by examining the slack bus row in the admittance matrix and computing the imaginary component of current flowing out, multiplied by the negative voltage angle of the connected bus. This class provides an efficient, structured approach for quickly estimating power flows and slack generation in a high-level power system simulation context, serving as a valuable approximation tool alongside more detailed AC methods.

2.9.4 ThreePhaseFaultParameters

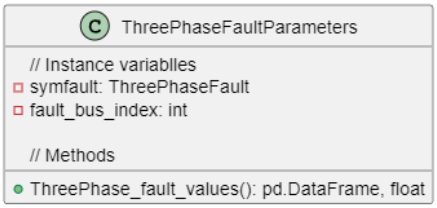


Figure 14: ThreePhaseFaultParameters Class

The `ThreePhaseFaultParameters` class is used to compute the electrical parameters resulting from a balanced three-phase (symmetrical) fault at a specified bus in the system. This analysis assumes the fault is balanced, meaning all three phases are shorted together with equal impedance, which allows for the use of positive-sequence components only. The `ThreePhase_fault_values()` method calculates the solution given the fault information

2.9.5 UnsymmetricalFaultParameters

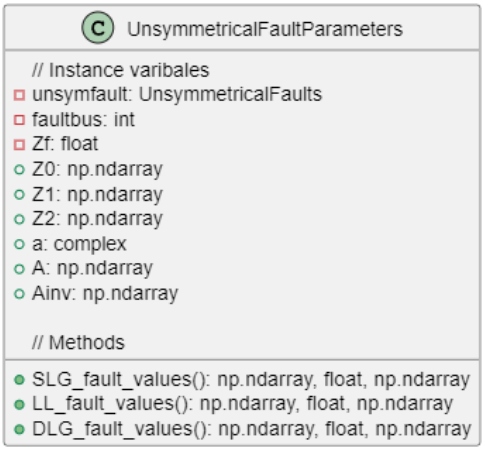


Figure 15: UnsymmetricalFaultParameters Class

The `UnsymmetricalFaultParameters` class is used to compute the electrical parameters resulting from unsymmetrical faults at a specified bus in the system. This class supports three common fault types: single line-to-ground (SLG), line-to-line (LL), and double line-to-ground (DLG). The class calculates the voltage profile across all buses in the system and determines the fault current and phase current at the faulted bus.

To initialize the class, the user must provide an `UnsymmetricalFaults` object containing the zero-, positive-, and negative-sequence impedance matrices, the index of the faulted bus, and the prefault voltage at that bus. An optional fault impedance can also be specified, which defaults to zero if not given. Internally, the class defines the symmetrical component transformation matrix $\mathbf{A}$, along with its inverse, to convert between sequence components and phase components.

The `SLG_fault_values()` method simulates a single line-to-ground fault and calculates the three-phase voltages at each bus during the fault. It also computes the fault current at the faulted bus, which is equal to three times the sequence current, and derives the corresponding phase current using the inverse symmetrical component transformation. The total impedance used for the SLG fault calculation includes the sum of the self-impedances of the zero-, positive-, and negative-sequence networks in addition to three times the fault impedance.

The `LL_fault_values()` method simulates a line-to-line fault, such as a B-C fault, and calculates the resulting voltages and currents. Because a line-to-line fault does not involve ground, there is no zero-sequence component. The method calculates the loop current from the difference between the positive- and negative-sequence contributions using the series impedance of the positive- and negative-sequence networks along with the fault impedance. It then transforms the sequence voltages to phase voltages and computes the fault and phase currents accordingly.

The `DLG_fault_values()` method simulates a double line-to-ground fault, such as a B-C-G fault. This is the most complex of the unsymmetrical faults, involving all three sequence networks interconnected in parallel. The method solves for the sequence currents by accounting for the mutual dependence between them and then transforms the sequence voltages back to phase components to calculate the bus voltages during the fault. The total fault current is taken as three times the zero-sequence current, and the corresponding phase current is calculated for the faulted bus.

All three methods return the bus voltage profile as a NumPy array of complex numbers with dimensions $(N, 3)$, where N is the number of buses and each row represents the three-phase voltages at that bus. Small numerical values are rounded to zero to improve readability. This class enables detailed unsymmetrical fault analysis, which is essential for power system protection, reliability studies, and proper relay coordination.

2.10 Tools

The Tools.py script defines utility functions for numerical operations and data handling in power system analysis. The `custom_round` function adjusts rounding based on the magnitude of numbers, while `custom_round_complex` applies this to complex numbers. The `to_csv` function formats complex numbers into a CSV file, and `read_excel` loads data from an Excel file, converting it to complex values. The `compare` function compares two matrices, `Ybus` and `pwrworld`, and displays their difference. The `read_jacobian` function extracts sub-matrices from a CSV file containing the Jacobian matrix of a power system, which is then used in further calculations. These functions support efficient data processing, matrix manipulation, and complex number handling for power flow analysis.

2.11 Transformer

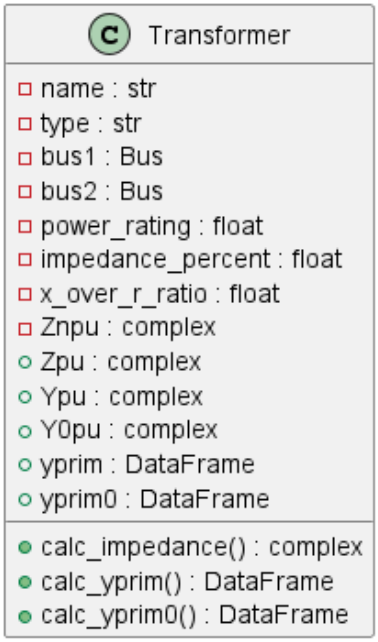


Figure 16: Transformer Class

The `Transformer` class models a power transformer within the simulation. It stores essential parameters provided during instantiation, including its `name`, connection `type` (e.g., Y-Y, Y-D), connected `Bus` objects (`bus1`, `bus2`), `power_rating`, `impedance_percent`, `x_over_r_ratio`, and optional neutral grounding impedance (`gnd_impedance`).

Upon initialization, it calculates the transformer's per-unit impedance (`Zpu`) based on its ratings and the system power base using the `calc_impedance()` method. This involves converting the given impedance percentage and X/R ratio into a complex per-unit value adjusted to the system's MVA base (`settings.powerbase`).

The class then determines the transformer's contribution to the system admittance matrix. The `calc_yprim()` method computes the 2x2 primitive admittance matrix (`yprim`) for the positive and negative sequence networks, representing the connection between `bus1` and `bus2`. This matrix is indexed by the bus indices and is suitable for direct inclusion in the system's positive/negative sequence Ybus.

Crucially for unbalanced fault analysis, the `calc_yprim0()` method calculates the zero-sequence primitive admittance matrix (`yprim0`). This calculation is dependent on the transformer's winding connection `type` (specified as "Y-Y", "Y-D", "D-Y", or "D-D") and the specified neutral grounding impedance (`Znpu`). It models how zero-sequence currents can flow through or are blocked by the transformer configuration, determining the appropriate entries for the zero-sequence Ybus. If no grounding impedance is provided (`Znpu` is `None`), the zero-sequence admittance contribution may be zero depending on the connection type.

2.12 TransmissionLine

Figure 17: TransmissionLine Class

The `TransmissionLine` class represents a power transmission line connecting two buses within the electrical network model. It encapsulates the line's physical and electrical characteristics. Instances can be created in two primary ways.

The standard constructor `__init__()` takes the line's `name`, the connecting `Bus` objects (`bus1`, `bus2`), and optionally requires detailed physical parameters: a `Bundle` object (containing conductor type and bundling configuration), a `Geometry` object (describing the physical spacing of the phase bundles), and the line's `length` in miles. When these physical details are provided, the constructor proceeds to calculate the line's per-unit series resistance (`R`), series reactance (`X`), and total shunt susceptance (`B`) using the helper methods `calc_R()`, `calc_X()`, and `calc_B()`, respectively. These calculations utilize conductor data, bundling details (GMR/DSC), phase spacing (Deq), line length, system frequency, and the calculated impedance base (`Zbase`) at the line's voltage level.

Alternatively, the class method `from_parameters()` allows for instantiation by directly providing the per-unit values of series resistance (`R`), series reactance (`X`), and total shunt susceptance (`B`), bypassing the need for detailed physical `Bundle` and `Geometry` objects and the associated calculations.

Regardless of the instantiation method, the class calculates the total series impedance (`Zseries`) and corresponding series admittance (`Yseries`). It also determines the primitive admittance matrices needed for constructing the system Ybus matrix. The `calc_yprim()` method generates the 2x2 primitive admittance matrix (`yprim`) for the

positive and negative sequence networks, based on a nominal-pi model using the calculated `Yseries` and `Yshunt`. Similarly, `calc_yprim0()` computes the zero-sequence primitive admittance matrix (`yprim0`), which is essential for unbalanced system studies. The zero-sequence calculation utilizes an approximated zero-sequence series admittance (`Y0series`) derived within the class. Both primitive matrices are returned as pandas DataFrames indexed by the connecting bus indices.

2.13 Validations

The `PowerBusSystemValidation.py` module defines functions for creating and validating power bus systems. Two test systems are implemented: a five-bus system and a seven-bus system, each with buses, transformers, transmission lines, generators, and loads. The validation functions ensure the accuracy of various system components, including impedance, admittance, Y-bus matrix, power flow algorithms (Newton-Raphson, Fast Decoupled, DC Power Flow), and fault analysis. For some validations, the module compares computed results against reference data from csv files, ensuring correctness in power system calculations. Specific validation functions include impedance checks, Y-bus validation, and fault analysis using the system's fault impedance matrix. The module is structured for use in power flow studies and fault analysis in electrical networks.

3 Relevant Equations

Circuit Building Equations

$$R = \frac{Z\% \cdot \cos\theta}{100} \tag{1}$$

$$X = \frac{Z\% \cdot \sin\theta}{100} \tag{2}$$

Series resistance and reactance of the transformer are found from $Z\%$ defined in given data for the transformer as well as the real or imaginary part of theta. This impedance is converted to per-unit in equation (4) for use in finding the system admittance matrix.

$$\theta = \tan^{-1}\left(\frac{X}{R}\right) \tag{3}$$

Theta is used in the calculation of series resistance and reactance for transformers. By taking the inverse tangent of the X/R ratio, equations (1) and (2) can be utilized later to find these values. Notably, shunt admittance for the transformer is ignored.

$$Z_{pu} = \frac{Z_{\text{actual}} \cdot S_{\text{base}}}{S_{\text{rated}}} \tag{4}$$

Per-unit impedance for a transformer can be found by multiplying the actual impedance of the transformer with the ratio of the MVA base of the system and the rated MVA of the equipment. In this simulator, the per-unit impedance is calculated separately for resistance and reactance and ultimately converted to a complex number.

$$Y = \frac{1}{Z} = \frac{1}{R + jX} \tag{5}$$

Admittance in general can simply be found by taking the reciprocal of an impedance. For the case of a transformer, the reciprocal of the per-unit impedance is useful for equation (6) and ultimately used in equation (19).

$$Y_{\text{prim}} = \begin{bmatrix} Y_{pu} & -Y_{pu} \\ -Y_{pu} & Y_{pu} \end{bmatrix} \tag{6}$$

The primitive matrix is useful for finding the admittance matrix of the entire system in equation (19). Each row and column has an associated bus that identifies how to add the per-unit values based on equations (20) and (21).

$$D_{\text{sc}} = r \tag{7}$$

$$D_{\text{sc}} = \sqrt{d \cdot r} \tag{8}$$

$$D_{\text{sc}} = \sqrt[3]{d^2 \cdot r} \tag{9}$$

$$D_{\text{sc}} = 1.091 \cdot \sqrt[4]{d^3 \cdot r} \tag{10}$$

Equations (7), (8), (9), and (10) are used based on whether the number of bundles is one, two, three, or four. Equation (7) is used for a bundle of one, equation (8) for a bundle of two, equation (9) for three, and equation (10) for four. These equations are useful in determining the shunt admittance of a transmission line given in equation (18). Note that the variable d is given by the outside diameter of the cable in inches.

$$D_{\text{sl}} = \text{GMR} \tag{11}$$

$$D_{\text{sl}} = \sqrt{d \cdot \text{GMR}} \tag{12}$$

$$D_{\text{sl}} = \sqrt[3]{d^2 \cdot \text{GMR}} \tag{13}$$

$$D_{\text{sl}} = 1.091 \cdot \sqrt[4]{d^3 \cdot \text{GMR}} \tag{14}$$

Equations (11), (12), (13), and (14) are used based on whether the number of bundles is one, two, three, or four. Equation (11) is used for a bundle of one, equation (12) for a bundle of two, equation (13) for three, and equation (14) for four. These equations are useful in determining the series reactance of a transmission line given in equation (17). Note that the variable GMR is the geometric mean radius of the cable in feet.

$$D_{\text{eq}} = \sqrt[3]{D_{ab} \cdot D_{bc} \cdot D_{ca}} \tag{15}$$

The D_{eq} value shown in equation (15) is used in both the calculation for series reactance in equation (17) and shunt admittance in equation (18). The variables D_{ab}, D_{bc}, and D_{ca} represent the geometric distance in feet between the cables of each of the three phases.

$$R = \frac{R_c \cdot l}{n} \tag{16}$$

To find the series resistance of a transmission in equation (16), three variables must be known. First, the resistance per conductor per mile, given by R_c is required. Then, a length given by l and a bundle number given by n allow the calculation of the line's series resistance.

$$X = 2\pi f \cdot 1609.34 \cdot l \cdot 2 \times 10^{-7} \cdot \ln\left(\frac{D_{\text{eq}}}{D_{\text{sl}}}\right) \tag{17}$$

The series reactance of a transmission line can be found with previously calculated D_{eq} and D_{sl}. In addition, the operating frequency of the system and the series length of the line must be taken into account when determining the reactance.

Several constant factors are also multiplied into this equation, all of which are very similar to the factors found in equation (18). First, the 2×10^{-7} factor comes from the vacuum magnetic permeability divided by 2π. Then, the 1609.34 factor comes from a meter-to-mile conversion. This is for the convenience of keeping the variable l in terms of miles. Finally, the 2πf factor is representative of the angular frequency of the system, which is multiplied by the implicitly calculated inductance L to determine the series reactance.

$$B = \frac{2\pi f \cdot 1609.34 \cdot l \cdot 2\pi\epsilon_0}{\ln\left(\frac{D_{\text{eq}}}{D_{\text{sc}}}\right)} \tag{18}$$

The shunt admittance of a transmission line can be found with previously calculated D_{eq} and D_{sc}. In addition, the operating frequency of the system and the series length of the line must be taken into account when determining the admittance.

Several constant factors are also multiplied in this equation, all of which are very similar to the factors found in Equation (17). First, the factor $2\pi\epsilon_0$ comes from the permittivity of free space multiplied by 2π. Then, the 1609.34 factor comes from a meter-to-mile conversion. This is for convenience in keeping the variable l in terms of miles.

Finally, the 2πf factor is representative of the angular frequency of the system, which is multiplied by the implicitly calculated capacitance C to determine the admittance of the shunt.

Admittance Matrix

$$Y_{\text{bus}} = \begin{bmatrix} Y_{\text{bus}}(1,1) & \dots & Y_{\text{bus}}(N,1) \\ \vdots & \ddots & \vdots \\ Y_{\text{bus}}(1,N) & \dots & Y_{\text{bus}}(N,N) \end{bmatrix} \tag{19}$$

The admittance matrix of the system is constructed from utilizing Equations (20) and (21). This matrix is later used to calculate the power flow solutions in the system. It is also used to find the impedance matrix later for fault analysis.

$$Y_{\text{bus}}(n,n) = \sum \text{pu admittances on bus } n \tag{20}$$

$$Y_{\text{bus}}(n,m) = -\sum \text{pu admittances from } n \text{ and } m \quad \text{for } n \neq m \tag{21}$$

To fill in the Y_{bus} in Equation (19), both the diagonal and off-diagonal elements can be found using separate algorithms. Using Equation (20) and summing the admittances connected to each bus, the diagonal elements of the bus admittance matrix can be found. Similarly, following the algorithm in Equation (21) allows for off diagonal elements to be calculated by summing admittances connecting two buses.

Power Mismatch and Injection

$$P_k = P_{G_k} - P_{L_k} \tag{22}$$

$$Q_k = Q_{G_k} - Q_{L_k} \tag{23}$$

Equations (22) and (23) imply that the complex power at each bus can be found from generated and load powers. With this, the power across the system can be initialized with each PQ bus.

$$P_k = \sum_{n=1}^{N} V_k Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn}) \tag{24}$$

$$Q_k = \sum_{n=1}^{N} V_k Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn}) \tag{25}$$

These equations define the real power and reactive power injected at each bus in terms of the system voltages, admittances, and angles. They are derived from the polar form of complex power flow and are essential for computing power mismatches in the Newton-Raphson power flow method. This is part of Step 1 in the Newton-Raphson algorithm.

$$\Delta \mathbf{y}(i) = \begin{bmatrix} \Delta \mathbf{P}(i) \\ \Delta \mathbf{Q}(i) \end{bmatrix} = \begin{bmatrix} \mathbf{P} - \mathbf{P}[\mathbf{x}(i)] \\ \mathbf{Q} - \mathbf{Q}[\mathbf{x}(i)] \end{bmatrix} \tag{26}$$

This vector represents the power mismatches at iteration i of the Newton-Raphson method. $\mathbf{P}$(i) and $\mathbf{Q}$(i) are the calculated powers at the current iteration, and the differences from the specified powers $\mathbf{P}$ and $\mathbf{Q}$ indicate the convergence error. These mismatches are used to update the state variables in the next iteration. This is part of Step 1 in the Newton-Raphson algorithm.

Newton-Raphson Algorithm

$$\mathbf{J} = \begin{bmatrix} \mathbf{J}_1 & \mathbf{J}_2 \\ \mathbf{J}_3 & \mathbf{J}_4 \end{bmatrix} \tag{27}$$

Note: $J_1 = \frac{\partial P}{\partial \delta}$, $J_2 = \frac{\partial P}{\partial V}$, $J_3 = \frac{\partial Q}{\partial \delta}$, $J_4 = \frac{\partial Q}{\partial V}$

This matrix is the Jacobian used in the Newton-Raphson method. Each submatrix represents partial derivatives of power equations with respect to voltage magnitude and angle. It quantifies how changes in the state variables affect the power mismatches, and it is updated at each iteration to guide convergence. Calculating the Jacobian is Step 2 of the Newton-Raphson algorithm.

$$J1_{kn} = \frac{\partial P_k}{\partial \delta_n} = V_k Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn}) \tag{28}$$

$$J2_{kn} = \frac{\partial P_k}{\partial V_n} = V_k Y_{kn} \cos(\delta_k - \delta_n - \theta_{kn}) \tag{29}$$

$$J3_{kn} = \frac{\partial Q_k}{\partial \delta_n} = -V_k Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn}) \tag{30}$$

$$J4_{kn} = \frac{\partial Q_k}{\partial V_n} = V_k Y_{kn} \sin(\delta_k - \delta_n - \theta_{kn}) \tag{31}$$

These expressions define the off-diagonal elements of the Jacobian submatrices in terms of bus voltages, admittance magnitudes, and phase angle differences. They are used to populate the full Jacobian matrix required for solving the nonlinear power flow equations using the Newton-Raphson method.

$$J1_{kk} = \frac{\partial P_k}{\partial \delta_k} = -\sum_{\substack{n=1 \\ n \neq k}}^{N} V_k Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn}) \tag{32}$$

$$J2_{kk} = \frac{\partial P_k}{\partial V_k} = V_k Y_{kk} \cos\theta_{kk} + \sum_{n=1}^{N} Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn}) \tag{33}$$

$$J3_{kk} = \frac{\partial Q_k}{\partial \delta_k} = V_k \sum_{\substack{n=1 \\ n \neq k}}^{N} Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn}) \tag{34}$$

$$J4_{kk} = \frac{\partial Q_k}{\partial V_k} = -V_k Y_{kk} \sin\theta_{kk} + \sum_{n=1}^{N} Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn}) \tag{35}$$

Similar to the previous set of equations, these expressions define the on-diagonal elements of the Jacobian submatrices in terms of bus voltages, admittance magnitudes, and phase angle differences.

$$\begin{bmatrix} \mathbf{J1}(i) & \mathbf{J2}(i) \\ \mathbf{J3}(i) & \mathbf{J4}(i) \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{\delta}(i) \\ \Delta\mathbf{V}(i) \end{bmatrix} = \begin{bmatrix} \Delta\mathbf{P}(i) \\ \Delta\mathbf{Q}(i) \end{bmatrix} \tag{36}$$

$$\begin{bmatrix} \Delta\boldsymbol{\delta}(i) \\ \Delta\mathbf{V}(i) \end{bmatrix} = \begin{bmatrix} \mathbf{J1}(i) & \mathbf{J2}(i) \\ \mathbf{J3}(i) & \mathbf{J4}(i) \end{bmatrix}^{-1} \begin{bmatrix} \Delta\mathbf{P}(i) \\ \Delta\mathbf{Q}(i) \end{bmatrix} \tag{37}$$

By finding the inverse of the solved Jacobian matrix for the current iteration as well as the power mismatch calculated earlier, the change in voltage magnitude and angle can be found according to Equations (36) and (37). This constitutes Step 3 for the Newton-Raphson algorithm.

$$\mathbf{x}(i+1) = \begin{bmatrix} \boldsymbol{\delta}(i+1) \\ \mathbf{V}(i+1) \end{bmatrix} = \begin{bmatrix} \boldsymbol{\delta}(i) \\ \mathbf{V}(i) \end{bmatrix} + \begin{bmatrix} \Delta\boldsymbol{\delta}(i) \\ \Delta\mathbf{V}(i) \end{bmatrix} \tag{38}$$

Equation (38) is the final step of the Newton-Raphson algorithm for the current iteration. This step is simply adding the result from Step 3 to the current voltage magnitude and angle. Each of these steps are completed in our simulation until a user-desired tolerance is met.

Fast Decoupled and DC Power Flow Algorithms

$$\mathbf{J}_1 \Delta\boldsymbol{\delta}(i) = \Delta\mathbf{P}(i) \tag{39}$$

$$\mathbf{J}_4 \Delta\mathbf{V}(i) = \Delta\mathbf{Q}(i) \tag{40}$$

The Fast Decoupled algorithm is similar to the Newton-Raphson algorithm. The primary difference between this algorithm and the Newton-Raphson algorithm is that only the first and fourth Jacobian matrices are considered. Following the equations above, the Fast Decoupled algorithm quickly approximates the Newton-Raphson solution.

$$P_{jk} = \frac{\delta_j - \delta_k}{X_{jk}} \tag{41}$$

$$-\mathbf{B}\boldsymbol{\delta} = \mathbf{P} \tag{42}$$

$$\boldsymbol{\delta} = -(\mathbf{B})^{-1}\mathbf{P} \tag{43}$$

The DC Power Flow algorithm relies on the imaginary part of the admittance matrix for calculations. It is primarily used to determine voltage angles and can be quickly solved similar to the Fast Decoupled algorithm via various approximations.

Three-Phase Fault

$$\mathbf{Z}_{\text{bus}} = \mathbf{Y}_{\text{bus}}^{-1} \tag{44}$$

$$E_k = (1 - \frac{Z_{kn}}{Z_{nn}})V_F \tag{45}$$

$$I''_{Fn} = \frac{V_F}{Z_{nn}} \tag{46}$$

Using these equations, three-phase faults (or symmetrical faults) can be solved for. By simply using the calculated admittance matrix, the impedance matrix is just the inverse. With the assumption of the pre-fault voltage being 1.0 pu, the sub-transient fault current and bus voltages can be found as well.

Symmetrical Components and Sequence Networks

$$a = e^{j\frac{2\pi}{3}} = 1\angle 120^\circ \tag{47}$$

$$\begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \\ 1 & a & a^2 \end{bmatrix} \begin{bmatrix} V_0 \\ V_1 \\ V_2 \end{bmatrix} \tag{48}$$

This can be written in compact vector form as:

$$\mathbf{V}_P = \mathbf{A}\mathbf{V}_S \tag{49}$$

$$\begin{bmatrix} V_0 \\ V_1 \\ V_2 \end{bmatrix} = \frac{1}{3}\begin{bmatrix} 1 & 1 & 1 \\ 1 & a & a^2 \\ 1 & a^2 & a \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} \tag{50}$$

In compact vector form:

$$\mathbf{V}_S = \mathbf{A}^{-1}\mathbf{V}_P \tag{51}$$

These equations are relevant to analysis of asymmetrical faults. Specifically, these equations relate to the method of symmetrical components. With these equations, zero, positive, and negative-sequence components can be determined from phase components. Conversely, the phase components can also be determined from the sequence components.

$$\mathbf{I}_P = \mathbf{A}\mathbf{I}_S \tag{52}$$

$$\mathbf{I}_S = \mathbf{A}^{-1}\mathbf{I}_P \tag{53}$$

Similarly, the current can be transformed between phase and sequence components. This follows the same logic as transformations with voltage.

Finding the zero, positive, and negative-sequence impedances is also critical for asymmetrical fault analysis. To accomplish this, separate analysis must be done for transmission lines, machines, and transformers. In particular, transformers will vary in their impedance depending on the type of connection being delta or wye.

$$Z_{0pu} \approx 2.5 \times (R_{pu} + jX_{pu}) \tag{54}$$

$$Z_{2pu} = Z_{1pu} = R_{pu} + jX_{pu} \tag{55}$$

For transmission lines, the sequence network can be sufficiently approximated using the equations above. Note that the positive and negative-sequence impedances remain unchanged whereas zero-sequence experiences a change in impedance. This is a common theme seen in machines and transformers.

$$Y_{g0} = \frac{1}{Z_{g0} + 3Z_n} \tag{56}$$

For machines, the primitive zero-sequence admittance matrix changes based on the equation above. This depends on a rated zero-sequence impedance and the grounding impedance. The positive and negative-sequence impedance are defined by the user.

$$Z_{0T,internal} = Z_{T,pu} + 3Z_{n,pu} \tag{57}$$

$$\begin{bmatrix} I_{0P} \\ I_{0S} \end{bmatrix} = \begin{bmatrix} Y_{0T,internal} & -Y_{0T,internal} \\ -Y_{0T,internal} & Y_{0T,internal} \end{bmatrix} \begin{bmatrix} V_{0P} \\ V_{0S} \end{bmatrix} \quad \text{(Y-Y)} \tag{58}$$

$$\begin{bmatrix} I_{0P} \\ I_{0S} \end{bmatrix} = \begin{bmatrix} Y_{0T,internal} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_{0P} \\ V_{0S} \end{bmatrix} \quad \text{(Y-D)} \tag{59}$$

$$\begin{bmatrix} I_{0P} \\ I_{0S} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & Y_{0T,internal} \end{bmatrix} \begin{bmatrix} V_{0P} \\ V_{0S} \end{bmatrix} \quad \text{(D-Y)} \tag{60}$$

$$\begin{bmatrix} I_{0P} \\ I_{0S} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_{0P} \\ V_{0S} \end{bmatrix} \quad \text{(D-D)} \tag{61}$$

Depending on the connection type for the transformer, the primitive admittance matrix varies. For wye-wye, it remains unchanged, whereas delta-delta disconnects it completely. With all of these sequence networks, the asymmetrical faults can be determined.

Asymmetrical Faults

$$I_{n-0} = I_{n-1} = I_{n-2} = \frac{V_F}{Z_{nn-0} + Z_{nn-1} + Z_{nn-2} + 3Z_F} \tag{62}$$

For single line-to-ground faults, the fault only affects one phase through a fault impedance Z_F.

$$I_{n-1} = -I_{n-2} = \frac{V_F}{Z_{nn-1} + Z_{nn-2} + Z_F} \tag{63}$$

$$I_{n-0} = 0 \tag{64}$$

For line-to-line faults, the fault is short between two phases between some fault impedance Z_F.

$$I_{n-1} = \frac{V_F}{Z_{nn-1} + \frac{Z_{nn-2}(Z_{nn-0}+3Z_F)}{Z_{nn-2}+Z_{nn-0}+3Z_F}} \tag{65}$$

$$I_{n-2} = -I_{n-1}\left(\frac{Z_{nn-0} + 3Z_F}{Z_{nn-0} + Z_{nn-2} + 3Z_F}\right) \tag{66}$$

$$I_{n-0} = -I_{n-1}\left(\frac{Z_{nn-2}}{Z_{nn-0} + Z_{nn-2} + 3Z_F}\right) \tag{67}$$

For double line-to-ground faults, the fault affects two phases passing through the same fault impedance Z_F.

$$\begin{bmatrix} V_{k-0} \\ V_{k-1} \\ V_{k-2} \end{bmatrix} = \begin{bmatrix} 0 \\ V_F \\ 0 \end{bmatrix} - \begin{bmatrix} Z_{kn-0} & 0 & 0 \\ 0 & Z_{kn-1} & 0 \\ 0 & 0 & Z_{kn-2} \end{bmatrix} \begin{bmatrix} I_{n-0} \\ I_{n-1} \\ I_{n-2} \end{bmatrix} \tag{68}$$

With the calculated fault currents, the voltage at a given bus can be solved using the fault current, pre-fault voltage, and impedance matrix information.

$$\begin{bmatrix} V_{ak} \\ V_{bk} \\ V_{ck} \end{bmatrix} = \mathbf{A} \begin{bmatrix} V_{k-0} \\ V_{k-1} \\ V_{k-2} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \\ 1 & a & a^2 \end{bmatrix} \begin{bmatrix} V_{k-0} \\ V_{k-1} \\ V_{k-2} \end{bmatrix} \tag{69}$$

$$\begin{bmatrix} I''_{an} \\ I''_{bn} \\ I''_{cn} \end{bmatrix} = \mathbf{A} \begin{bmatrix} I''_{n-0} \\ I''_{n-1} \\ I''_{n-2} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & a^2 & a \\ 1 & a & a^2 \end{bmatrix} \begin{bmatrix} I_{n-0} \\ I_{n-1} \\ I_{n-2} \end{bmatrix} \tag{70}$$

To find the phase voltages at different buses, the above transformation can be used. Likewise, the same method can be used to solve for subtransient currents.

4 Example Case

The following system will be used as an example to showcase the functionality of our simulator. We have seven buses, two transformers, two generators, three loads, and various transmission lines.

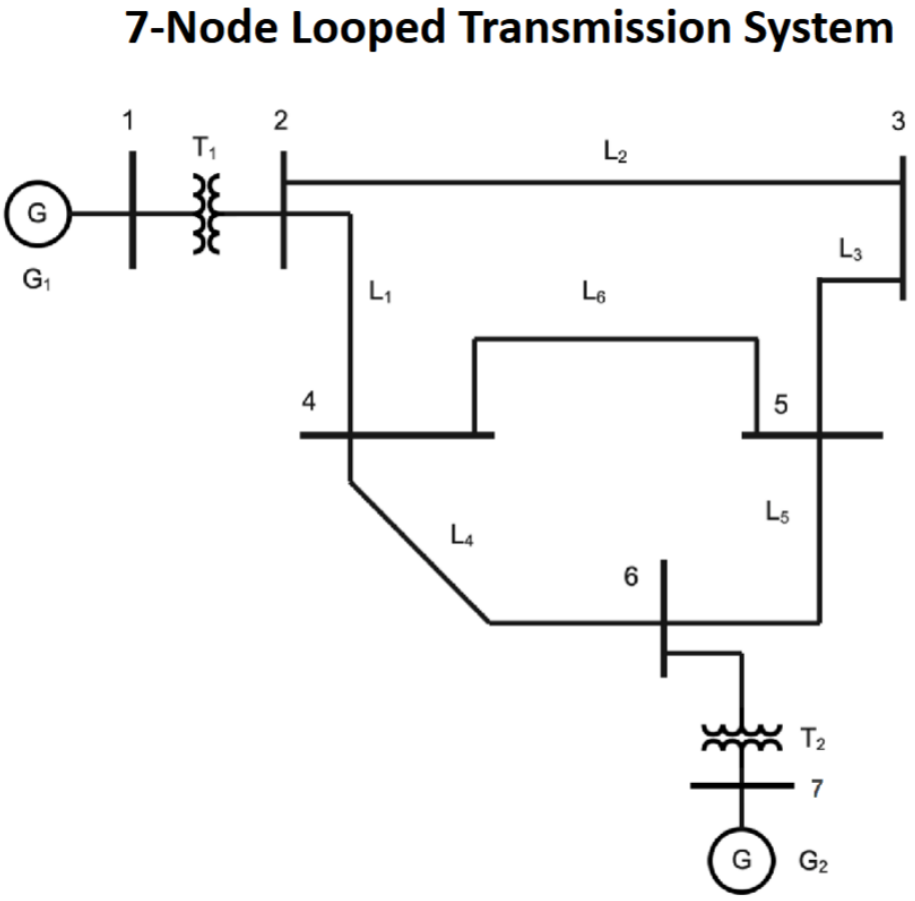


Figure 18: Seven bus system

ACSR Conductor Type

Partridge
Bundles of 2 subconductors per phase with 1.5' spacing

Line Lengths

L1: 10 miles
L2: 25 miles
L3: 20 miles
L4: 10 miles
L5: 35 miles

Transformers

T1: 125 MVA, 20 kVΔ/230 kV-Y,
z = 0.085 pu, x/r = 10
Grounded with a 1-ohm grounding impedance on the wye side

T2: 200 MVA, 18 kVΔ/230 kV-Y,
z = 0.105 pu, x/r = 12
Wye connection is ungrounded

Line Structure Diagram

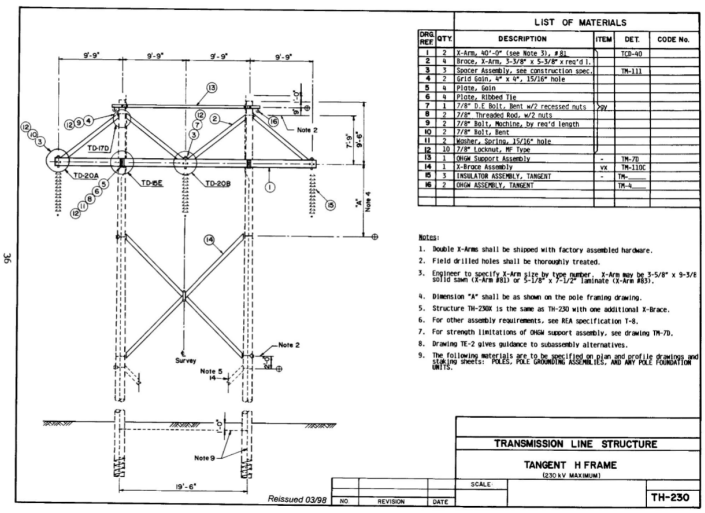


Figure 19: Seven bus system data

4.1 Problem Definition

Given the above system, three different power flow algorithms are used to find the voltages at each bus and the power injected by each generator.

4.2 Solution Process

The main.py file functions as a blank canvas where the user can begin creating circuits. Every circuit object needs to be initialized with a name. Since this system uses seven buses, we'll give it the name "Example_7bus" and the actual Circuit object will be called 'circ' for short.

```
circ = Circuit("Example_7bus")
```

Figure 20: Creating a circuit object

Creating a circuit object automatically sets the power base internally, which by default is 100 MVA. To change the system's base power, use the Circuit classes *change_power_base* method before creating any components. To show how this would be done, we'll simply change the base power to 100 MVA. The software internally converts the chosen number to MVA.

```
circ.change_power_base(100)
```

Figure 21: Changing system base power

Now, we'll add our buses. To create a bus object, pass a name and voltage value (in kV) to the *add_bus* method. The software internally handles the conversion to kilovolts.

```
circ.add_bus("bus1", 20)
circ.add_bus("bus2", 230)
circ.add_bus("bus3", 230)
circ.add_bus("bus4", 230)
circ.add_bus("bus5", 230)
circ.add_bus("bus6", 230)
circ.add_bus("bus7", 18)
```

Figure 22: Creating buses

Using the data from Table 1, we use the *add_transformer* method to create our transformers. Note, "D-Y" and "Y-D represent Delta-Wye and Wye-Delta connections, respectively. Also, since T2 is ungrounded, we don't enter any value for its ground impedance.

Name	Connection	Bus A	Bus B	Rating (MVA)	Impedance Percent	X/R Ratio	Ground Impedance (pu)
T1	Delta-Wye	1	2	125	8.5	10	0.0018904
T2	Wye-Delta	6	7	200	10.5	12	None

Table 1: Transformer Data

```
circ.add_transformer("T1", "D-Y", "bus1", "bus2", 125, 8.5, 10, 0.0018904)
circ.add_transformer("T2", "Y-D", "bus6", "bus7", 200, 10.5, 12)
```

Figure 23: Creating transformers

Before creating the transmission lines, we need to create Conductor, Geometry, and Bundle objects. We'll use the Partridge conductor, which has an outside diameter of 0.642 inches, a GMR of 0.0217 ft, a resistance of 0.385 Ω/mi at 50°C, and an ampacity of 460 amps.

```
circ.add_conductor("Partridge", 0.642, 0.0217, 0.385, 460)
```

Figure 24: Creating conductor

We'll give our geometry the name "Geometry7bus". The transmission lines are placed 19.5 meters apart in the x-direction, going from 0 m to 19.5 m, and 19.5 m to 39 m. They all have the same y-coordinate of 0. Figure 26 shows how to create a Geometry object with this information.

```
circ.add_geometry("Geometry7bus", [0, 19.5, 39], [0, 0, 0])
```

Figure 25: Creating geometry

The system has 2 subconductors per phase with a 1.5-inch spacing between each conductor. Additionally, every bundle object should have a name. We'll give our bundle the name "Bundle7Bus". Pass the Bundle's name, number of subconductors, spacing, and the name of the conductor we previously created to the *add_bundle* method.

```
circ.add_bundle("Bundle7bus", 2, 1.5, "Partridge")
```

Figure 26: Creating bundle

The system has six transmission lines in total. Since this system has transmission lines created using Bundle and Geometry objects, we'll use the *add_tline_from_geometry* method.

Name	Bus A	Bus B	Length (Miles)
L1	bus2	bus4	10
L2	bus2	bus3	25
L3	bus3	bus5	20
L4	bus4	bus6	20
L5	bus5	bus6	10
L6	bus4	bus5	35

Table 2: Line Information

```
circ.add_tline_from_geometry("L1", "bus2", "bus4", "Bundle7bus", "Geometry7bus", 10)
circ.add_tline_from_geometry("L2", "bus2", "bus3", "Bundle7bus", "Geometry7bus", 25)
circ.add_tline_from_geometry("L3", "bus3", "bus5", "Bundle7bus", "Geometry7bus", 20)
circ.add_tline_from_geometry("L4", "bus4", "bus6", "Bundle7bus", "Geometry7bus", 20)
circ.add_tline_from_geometry("L5", "bus5", "bus6", "Bundle7bus", "Geometry7bus", 10)
circ.add_tline_from_geometry("L6", "bus4", "bus5", "Bundle7bus", "Geometry7bus", 35)
```

Figure 27: Creating transmission lines

The parameters for each generator are listed in Table 2. Gen1 is at the system's slack bus, which is connected to a 125 MVA transformer, so we'll set Gen1's power rating to 125 MW. Using the same logic for Gen2 and T2, we set Gen2's power rating to 200 MW. X1 = positive sequence impedance, X0 = zero sequence impedance, X2 = negative sequence impedance, and Zn = ground impedance. Since Gen1 is solidly grounded with a wire, we enter 0 for its ground impedance. We'll use the *add_generator* method along with the information from Table 3 to create our Generator objects.

Name	Bus	Voltage (pu)	Power (MW)	X1 (pu)	X0 (pu)	X2 (pu)	Zn (pu)
Gen1	bus1	1	125	0.12	0.14	0.05	0
Gen2	bus7	1	200	0.12	0.14	0.05	0.30864

Table 3: Generator information

```
circ.add_generator("Gen1", "bus1", 1, 125, 0.12, 0.14, 0.05, 0)
circ.add_generator("Gen2", "bus7", 1, 200, 0.12, 0.14, 0.05, 0.30864)
```

Figure 28: Creating generators

Finally, we'll add our loads.

Name	Bus	Real Power (MW)	Reactive Power (MVAR)
Load1	bus3	110	50
Load2	bus4	100	70
Load3	bus5	100	65

Table 4: Generator information

```
circ.add_load("Load1", "bus3", 110, 50)
circ.add_load("Load2", "bus4", 100, 70)
circ.add_load("Load3", "bus5", 100, 65)
```

Figure 29: Creating loads

4.3 Expected Outputs

Now that the system is created, we can solve for the bus voltages using the included power flow solvers. Since the Fast Decoupled solver produces the same results as the Newton-Raphson solver, we'll use the latter and the DC Power Flow solver. After a power solver is used, fault analysis can be done on a bus of the user's choice. Every output from this software will be shown next to a PowerWorld output for comparison.

4.3.1 Newton-Raphson Solver

The *do_newton_raph* method uses the Newton-Raphson algorithm to calculate the voltages and power injections at each bus. The software automatically calculates everything needed before the computations are done, and when the solver is finished, the results are printed to the output terminal, as seen in Figure 32. PowerWorld has this same algorithm implemented, and Figure 33 shows the results from PowerWorld.

```
circ.do_newton_raph()
```

Figure 30: Newton-Raphson Method

	Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	115.9	85.8
2	2	bus2	230.0	0.93692	215.492	-4.445	0.0	0.0	0.0	0.0
3	3	bus3	230.0	0.92049	211.712	-5.466	110.0	50.0	0.0	0.0
4	4	bus4	230.0	0.9298	213.854	-4.704	100.0	70.0	0.0	0.0
5	5	bus5	230.0	0.92673	213.147	-4.835	100.0	65.0	0.0	0.0
6	6	bus6	230.0	0.93968	216.127	-3.953	0.0	0.0	0.0	0.0
7	7	bus7	18.0	1.0	18.0	2.149	0.0	0.0	200.0	108.8

Figure 31: Our Newton-Raphson results

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			115.86	85.81
2	2	2	1	230.00	0.93692	215.491	-4.44				
3	3	3	1	230.00	0.92049	211.712	-5.46	110.00	50.00		
4	4	4	1	230.00	0.92980	213.853	-4.70	100.00	70.00		
5	5	5	1	230.00	0.92672	213.147	-4.83	100.00	65.00		
6	6	6	1	230.00	0.93968	216.126	-3.95				
7	7	7	1	18.00	0.99999	18.000	2.15			200.00	108.79

Figure 32: PowerWorld's Newton-Raphson results

4.3.2 DC Power flow Solver

Similarly, the DC Power Flow can be solved in our simulation. Below are the results of running this algorithm in our simulation as well as verification from PowerWorld.

	Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	113.76	0.0
2	2	bus2	230.0	1.0	230.0	-4.454	0.0	0.0	0.0	0.0
3	3	bus3	230.0	1.0	230.0	-5.612	110.0	0.0	0.0	0.0
4	4	bus4	230.0	1.0	230.0	-4.799	100.0	0.0	0.0	0.0
5	5	bus5	230.0	1.0	230.0	-4.958	100.0	0.0	0.0	0.0
6	6	bus6	230.0	1.0	230.0	-3.956	0.0	0.0	0.0	0.0
7	7	bus7	18.0	1.0	18.0	2.081	0.0	0.0	200.0	0.0

Figure 33: Our DC Power flow Results

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			110.00	0.00
2	2	2	1	230.00	1.00000	230.000	-4.26				
3	3	3	1	230.00	1.00000	230.000	-5.29	110.00	0.00		
4	4	4	1	230.00	1.00000	230.000	-4.56	100.00	0.00		
5	5	5	1	230.00	1.00000	230.000	-4.70	100.00	0.00		
6	6	6	1	230.00	1.00000	230.000	-3.79				
7	7	7	1	18.00	1.00000	18.000	2.20			200.00	0.00

Figure 34: PowerWorld's DC Power flow results

4.3.3 Three Phase Faults

Symmetrical fault analysis can be done by creating a ThreePhaseFault object, like shown in Figure 36. The circuit object we previously created has to be passed as an argument, along with an integer representing the bus we want to do fault analysis at.

```
symfault = ThreePhaseFault(circ, 1)
symfault.calc_fault_values()
```

Figure 35: Creating a ThreePhaseFault object

```
ThreePhase Current:
      Magnitude  Phase A Angle  Phase B Angle  Phase C Angle
Bus1     16.138         -84.35         155.65          35.65

ThreePhase Fault Voltages:
   Phase A  Phase B  Phase C  Phase A Angle  Phase B Angle  Phase C Angle
1  0.00000  0.00000  0.00000           0.00        -120.00         120.00
2  0.32669  0.32669  0.32669          -0.59        -120.59         119.41
3  0.36457  0.36457  0.36457          -2.49        -122.49         117.51
4  0.36707  0.36707  0.36707          -1.69        -121.69         118.31
5  0.40263  0.40263  0.40263          -2.40        -122.40         117.60
6  0.43644  0.43644  0.43644          -1.96        -121.96         118.04
7  0.73287  0.73287  0.73287           4.20        -115.80         124.20
```

Figure 36: Our Three Phase Fault results

Fault Current

Scale Current By: 1.00000

If Magnitude: 16.138 p.u.

If Scaled Mag: 16.138 p.u.

If Angle: -84.35 deg.

Units: (•) p.u. ( ) Amps

Subtransient Phase Current

	p.u.	deg.
A	16.138	-84.35
B	16.138	155.65
C	16.138	35.65

	Number	Name	Phase Volt A	Phase Volt B	Phase Volt C	Phase Ang A	Phase Ang B	Phase Ang C
1	1	1	0.00000	0.00000	0.00000	90.00	90.00	90.00
2	2	2	0.32669	0.32669	0.32669	-0.59	-120.59	119.41
3	3	3	0.36457	0.36457	0.36457	-2.49	-122.49	117.51
4	4	4	0.36707	0.36707	0.36707	-1.69	-121.69	118.31
5	5	5	0.40263	0.40263	0.40263	-2.40	-122.40	117.60
6	6	6	0.43644	0.43644	0.43644	-1.96	-121.96	118.04
7	7	7	0.73286	0.73286	0.73286	4.21	-115.79	124.21

Figure 37: PowerWorld's Three Phase Fault results

4.3.4 Single Line-to-Ground Faults

Setting up an unsymmetrical fault analysis is similar to the process used when doing symmetrical fault analysis. We create an UnsymmetricalFaults object, pass it a circuit object and a bus number, then call the method for whatever fault we want to do. Since we want to run Single Line-to-ground Fault analysis, we use the *SLG_fault_values* method. All faults will be done at bus 1.

```
unsymfault = UnsymmetricalFaults(circ, 1)
unsymfault.SLG_fault_values()
```

Figure 38: Creating a UnsymmetricalFaults object

```
Fault current: 17.555 pu
Subtransient Phase Current
   Magnitude(pu)  Angle(deg)
A         17.555      -85.37
B          0.000        0.00
C          0.000        0.00

   Phase A  Phase B  Phase C  Phase A Angle  Phase B Angle  Phase C Angle
1  0.00000  0.99588  0.93926           0.00        -110.89         111.53
2  0.46468  0.88467  0.85384          -0.83        -109.96         100.98
3  0.48921  0.87144  0.84376          -2.40        -112.06         101.04
4  0.49327  0.88022  0.85212          -1.62        -111.27         101.77
5  0.51920  0.87952  0.85360          -2.15        -112.29         102.53
6  0.54756  0.89344  0.86916          -1.58        -112.08         104.08
7  0.78432  0.98281  0.95386           4.17        -111.64         116.11
```

Figure 39: Our Single Line to Ground results

Fault Current

Scale Current By: 1.00000

If Magnitude: 17.555 p.u.

If Scaled Mag: 17.555 p.u.

If Angle: -85.37 deg.

Units: p.u. / Amps

Subtransient Phase Current

	p.u.	deg.
A	17.555	-85.37
B	0.000	-89.46
C	0.000	-89.46

	Number	Name	Phase Volt A	Phase Volt B	Phase Volt C	Phase Ang A	Phase Ang B	Phase Ang C
1	1	1	0.00000	0.99588	0.93926	1.79	-110.89	111.53
2	2	2	0.46468	0.88467	0.85383	-0.83	-109.96	100.98
3	3	3	0.48921	0.87144	0.84376	-2.40	-112.06	101.04
4	4	4	0.49327	0.88022	0.85212	-1.62	-111.27	101.77
5	5	5	0.51920	0.87952	0.85360	-2.15	-112.29	102.53
6	6	6	0.54756	0.89344	0.86916	-1.58	-112.08	104.08
7	7	7	0.78432	0.98281	0.95386	4.17	-111.64	116.11

Figure 40: PowerWorld's Single Line to Ground results

4.3.5 Line-to-Line Faults

Like the SLG faults, the LL faults can be simulated using an UnsymmetricalFaults object. In this case, the Line-to-Line Fault analysis will be called with the *LL_fault_values* method in Figure 42. Figures 43 and 44 show the results of the algorithm in our simulation as well as PowerWorld verification.

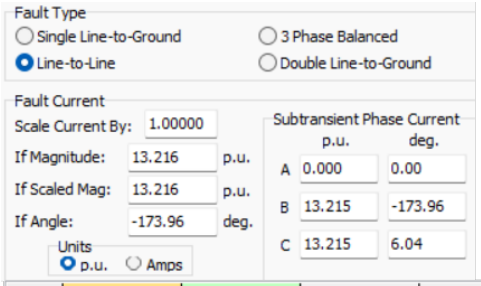
```
unsymfault.LL_fault_values()
```

Figure 41: Using the Line to Line method

```
Fault current: 13.216 pu
Subtransient Phase Current
   Magnitude(pu)  Angle(deg)
A          0.000        0.00
B         13.216     -173.96
C         13.216        6.04

   Phase A  Phase B  Phase C  Phase A Angle  Phase B Angle  Phase C Angle
1  1.05443  0.52721  0.52721          -0.35         179.65         179.65
2  0.97679  0.54233  0.58014          -4.81        -154.18         146.75
3  0.95817  0.55369  0.58635          -5.82        -151.94         142.41
4  0.96791  0.55862  0.59188          -5.06        -151.26         143.27
5  0.96346  0.57550  0.60521          -5.18        -148.84         140.51
6  0.97597  0.59899  0.62582          -4.30        -146.16         139.47
7  1.02947  0.78853  0.82537           1.81        -126.22         133.01
```

Figure 42: Our Line to Line results



	Number	Name	Phase Volt A	Phase Volt B	Phase Volt C	Phase Ang A	Phase Ang B	Phase Ang C
1	1	1	1.05443	0.52721	0.52721	-0.35	179.65	179.65
2	2	2	0.97679	0.54233	0.58014	-4.81	-154.18	146.75
3	3	3	0.95817	0.55368	0.58635	-5.82	-151.94	142.41
4	4	4	0.96791	0.55862	0.59188	-5.06	-151.26	143.26
5	5	5	0.96346	0.57550	0.60521	-5.19	-148.84	140.51
6	6	6	0.97597	0.59899	0.62582	-4.30	-146.16	139.47
7	7	7	1.02947	0.78853	0.82538	1.81	-126.22	133.01

Figure 43: PowerWorld's Line to Line results

4.3.6 Double Line-to-Ground Faults

Similar to the SLG and LL faults, the DLG fault analysis can be done with a call to the *DLG_fault_values* method in Figure 45 with a given UnsymmetricalFaults object. In Figures 46 and 47, the results of the simulation are verified by PowerWorld.

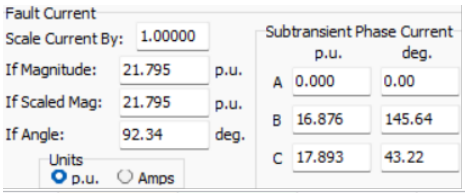
```
unsymfault.DLG_fault_values()
```

Figure 44: Using the Double Line to Ground method

```
Fault current: 21.795 pu
Subtransient Phase Current
   Magnitude(pu)  Angle(deg)
A          0.000       90.00
B         16.876      145.64
C         17.893       43.21

   Phase A  Phase B  Phase C  Phase A Angle  Phase B Angle  Phase C Angle
1  0.87178  0.00000  0.00000           2.34           0.00           0.00
2  0.68541  0.43464  0.44995          -2.50        -142.47         139.10
3  0.69212  0.46041  0.47160          -3.76        -141.09         134.81
4  0.69861  0.46409  0.47559          -2.99        -140.38         135.67
5  0.71209  0.49115  0.49999          -3.31        -138.74         133.11
6  0.73412  0.52043  0.52714          -2.58        -136.69         132.28
7  0.89693  0.76398  0.77848           3.18        -121.62         129.49
```

Figure 45: Our Double Line to Ground Results



	Number	Name	Phase Volt A	Phase Volt B	Phase Volt C	Phase Ang A	Phase Ang B	Phase Ang C
1	1	1	0.87178	0.00000	0.00000	2.34	135.00	0.00
2	2	2	0.68541	0.43464	0.44995	-2.50	-142.47	139.10
3	3	3	0.69212	0.46041	0.47160	-3.76	-141.09	134.81
4	4	4	0.69861	0.46409	0.47559	-2.99	-140.38	135.67
5	5	5	0.71209	0.49115	0.49999	-3.31	-138.74	133.11
6	6	6	0.73412	0.52043	0.52715	-2.58	-136.69	132.28
7	7	7	0.89693	0.76398	0.77848	3.18	-121.62	129.49

Figure 46: PowerWorld's Double Line to Ground Results

5 Project 3 Addition - Solar Generation

5.1 Solar Overview

Introduction

This section is dedicated to additions and changes made after the PV addition to this simulator. This PV simulation is integrated into the simulation in a modular manner, relying on previously built framework. The focus of this addition was for power flow analysis, but faults can still be studied just as they were in the previous simulation. Furthermore, fundamental architecture such as buses, generators, loads, transmission lines, and transformers all cooperate cleanly with the newly added solar class. The primary goal of this addition is to study how PV generation and load fluctuations impact power systems over time and how this leads to the Duck Curve.

Significant Additions and Changes

Notably, the most important change in with this version of the power simulator is the ability to analyze solar generation. Specifically, this is implemented via a Solar.py module. Within this module is a Solar class, constructed with a name, bus, real power, power factor, and leading/lagging/unity power factor specification. This simple architecture calculates reactive power based on target power factor and offers flexibility for user control. For instance, there are several functions to set variables as well as a __str__ for ease of observing how variables behave.

Another important change included in this version of the simulator is various changes in Circuit.py. Within the Circuit class, Solar objects can be added at specified buses and behave similar to a PQ type object, but instead of consuming power they inject power into the system. This is taken into account in the power flow solution, allowing for static one-time solutions to be solved with PV generation.

An important characteristic of solar generation is that it is not as dispatchable as a traditional generator modeled in this simulation - the power generation is dependent on irradiance. For this reason, functionality focused on variable irradiance over time has been integrated into Circuit.py. Within the simulation, an array of factors by which the power is multiplied by is included and approximately represents a bell curve. By varying this power over several iterations and solving the power flow of the system, analysis of solar generation under constant load over time can be performed.

To observe and study the Duck Curve, variable loads were to be included as well. The refactoring of the Load and Circuit classes required to realize this is fairly straightforward. Following the same logic for variable power as in the development of variable irradiance, the power demand of the loads change alongside the power generation of the PV generators.

Overall, this addition allows for study of how power systems vary over time with variable solar generation alongside variable demand over time. With this integration, data can be analyzed and studied to minimize dramatic peaks associated with the Duck Curve.

5.2 Class Diagram and Refactoring

Solar

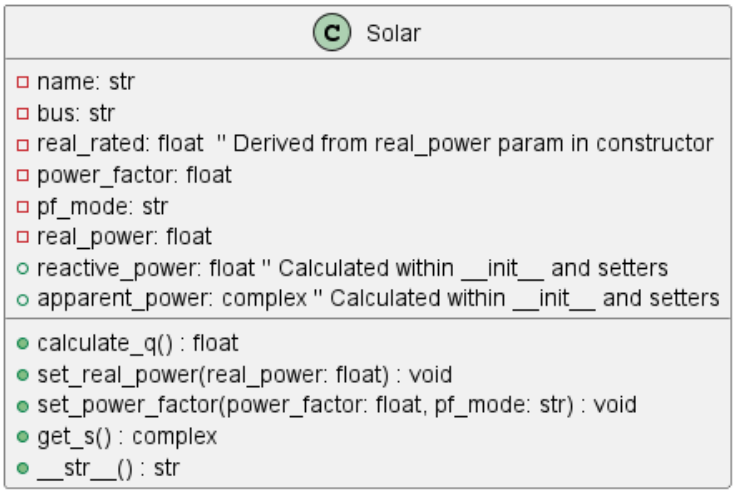


Figure 47: Solar Class

The `Solar` class provides a representation of a photovoltaic (PV) generator for use within the power system, specifically designed to integrate with the `Circuit` class. Its primary function is to model a solar power source operating in a constant power (PQ) mode, meaning it injects a specified amount of real power (P) and reactive power (Q) into the grid. Instances of the `Solar` class are created and managed by the `Circuit` object, which stores them in its `solar` dictionary attribute via the `add_solar` method. This integration allows the `Circuit` to account for the power injections from solar sources when performing power flow calculations or time-series simulations, such as those executed by the `sweep_solar` method.

Upon initialization, a `Solar` object requires several parameters to be passed through the constructor. The `name` parameter assigns a unique string identifier to the PV generator. The `bus` parameter specifies the name of the bus within the `Circuit` to which this generator is connected. The `real_power` parameter defines the rated or initial real power output of the generator in megawatts (MW). This value is stored internally both as (`real_rated`) and the current operating real power (`real_power`), converted to Watts for internal calculations. The constructor enforces that this input real power must be non-negative. The target power factor is set using the `power_factor` parameter, a value between 0 (exclusive) and 1 (inclusive), and the `pf_mode` parameter, a string indicating whether the power factor is `'leading'`, `'lagging'`, or `'unity'`. The constructor validates these inputs; the power factor must be within the specified range, and the mode must be one of the allowed strings. If `'unity'` mode is selected, the power factor is automatically set to 1.0, overriding any other value provided. Based on the initial real power, power factor, and mode, the constructor immediately calculates the corresponding reactive power (`reactive_power` in VARs) by calling the internal `calculate_q` method, and subsequently determines the complex apparent power (`apparent_power` in VA), storing these as internal attributes.

The internal `calculate_q` method is responsible for determining the reactive power output based on the current `real_power`, `power_factor`, and `pf_mode`. It handles edge cases where the real power is negligible or the power factor is unity, returning zero

reactive power in these scenarios. Otherwise, it calculates the magnitude of the reactive power using the relationship $Q = |P| \tan(\arccos(\text{pf}))$. The sign of Q is determined by the `pf_mode`: positive for `'lagging'` (generator supplies reactive power) and negative for `'leading'` (generator absorbs reactive power). This method is automatically invoked during initialization and whenever the real power or power factor settings are modified.

To allow dynamic changes to the generator's output, the class provides setter methods. The `set_real_power` method accepts a new real power value in MW, validates that it is non-negative, updates the internal `real_power` attribute (converting to Watts), and then automatically recalculates the `reactive_power` and `apparent_power` based on the existing power factor settings by calling `calculate_q`. Similarly, the `set_power_factor` method allows updating both the `power_factor` value and the `pf_mode` (restricted to `'lagging'` or `'leading'` in this setter). It validates the inputs, updates the corresponding attributes, and recalculates the `reactive_power` and `apparent_power`.

For retrieving the calculated complex power, the `get_s` method is provided. It returns the value of the `apparent_power` attribute, converted back to MVA for consistency with the input units and typical power system reporting.

Finally, the class implements the standard `__str__` method, which provides a concise, human-readable string representation of the `Solar` object. This string includes the generator's name, connected bus, and its current real power output (in MW) and reactive power output (in MVAR), facilitating easy inspection and debugging.

Refactors

The first refactor is for Component.py, focused on the addition of `real_rated` and `reactive_rated`. These are reference variables used to keep track of relative load over time. Specifically, this value matches the load's assigned power variables in the static case, but is used to calculate dynamic changing power, similar to the analogous variables in the `Solar` class.

Constants.py was also refactored to include arrays of normalized factors by which the `Load` and `Solar` object's rated powers can change over time. Specifically, `solar_profile` is multiplied with the rated power of every `Solar` object over 24 hour iterations. Likewise, `load_profile_factor` is multiplied to every `Load` object in the same manner.

Bus.py was refactored to allow for more flexible power flow assignment. Initially, `Bus` objects were rigid in their assignment for flat start, but with a time-varying system the class had to be changed. To allow for this, the `subtract_power` method was implemented and allows for assigned power to be removed.

Validations.py saw some changes to allow for testing pre-built scenarios involving the `Solar` class. These scenarios will be explored in the next section discussing results and verification with PowerWorld.

Circuit.py saw the most significant refactoring. First, during construction the class now initializes an attribute `solar_sweep_data` to hold information gathered during time-series simulation. Additionally, several functions were added:

- `add_solar(self, name, bus, real_power, power_factor, pf_mode)`
  - **Purpose:** Adds a new solar PV generator model to the circuit simulation.
  - **Parameters:**
    - `name` (str): Unique name for the solar generator.

    * `bus` (str): Name of the bus where the generator connects.
    * `real_power` (float): Rated real power output [MW].
    * `power_factor` (float): Operational power factor (0 to 1).
    * `pf_mode` (str): Power factor mode ('leading', 'lagging', 'unity').

- `sweep_solar(self, dynamic_load=False, csv=False)`
  - **Purpose:** Simulates the circuit over 24 hours, varying solar output (and optionally load) based on hourly profiles, and collects results.
  - **Parameters:**
    * `dynamic_load` (bool): If True, vary loads hourly too. Default: False.
    * `csv` (bool): If True, save simulation results to a CSV file. Default: False.

- `modify_solar(self, irradiance, restore_default=False)`
  - **Purpose:** Adjusts the power output of all solar generators based on an irradiance factor, or resets them to rated power. Typically used within `sweep_solar`.
  - **Parameters:**
    * `irradiance` (float): Factor (usually 0-1) to scale rated solar power.
    * `restore_default` (bool): If True, reset power to rated instead of applying irradiance. Default: False.

- `modify_load(self, factor, restore_default=False)`
  - **Purpose:** Adjusts the power consumption of all loads based on a scaling factor, or resets them to rated power. Typically used within `sweep_solar` if `dynamic_load` is True.
  - **Parameters:**
    * `factor` (float): Factor to scale rated load power.
    * `restore_default` (bool): If True, reset load to rated instead of applying factor. Default: False.

- `write_sweep_results_to_csv(self, filename="solar_sweep_results.csv")`
  - **Purpose:** Writes the hourly simulation results collected by `sweep_solar` to a CSV file.
  - **Parameters:**
    * `filename` (str): Name of the output CSV file. Default: "solar_sweep_results.csv".

5.3 Example Results and Verification

Scenario I: Static Seven Bus System

One of the refactors of Validations.py was `Testing_Scenario1`. This scenario consists of the standard seven bus system used in the example case in section four. However, this system has a PV generator attached to `bus7` rated for 50 MW at 0.95 lagging power factor.

```
def Testing_Scenario1():
    # Scenario 1: Static solar panel test with seven power bus system
    # and solar generation at bus7
    circ = CreateSevenPowerBusSystem()

    # Added PV generation at bus7 with 50 MW at 0.95 lagging power factor
    circ.add_solar("Solar_Scenario1", "bus7", 50, 0.95, "lagging")

    # NewtonRaphson algorithm performed on this circuit - compare with PowerWorld
    print(f"PROJECT 3 SCENARIO 1: TEST\n")
    NewtonRaphValidation(circ)
```

Listing 1: Python function for Scenario 1 test setup.

Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	66.83	90.65
2	bus2	230.0	0.93496	215.041	-2.396	0.0	0.0	0.0	0.0
3	bus3	230.0	0.91895	211.358	-3.138	110.0	50.0	0.0	0.0
4	bus4	230.0	0.92825	213.497	-2.382	100.0	70.0	0.0	0.0
5	bus5	230.0	0.92555	212.877	-2.278	100.0	65.0	0.0	0.0
6	bus6	230.0	0.93888	215.943	-1.216	0.0	0.0	0.0	0.0
7	bus7	18.0	1.0	18.0	6.491	0.0	0.0	250.0	112.19

Table 5: Newton-Raphson Power Flow Results for Scenario 1

From the table, we can see that the Gen MW on bus7 have increased by 50 MW, aligning with expectations of including a 50 MW static PV generator on bus7. To verify the rest of the table, PowerWorld can be used.

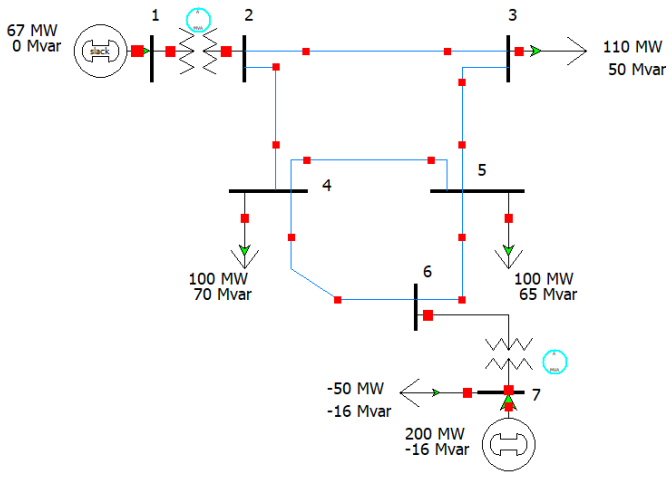


Figure 48: PowerWorld implementation of Scenario 1

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			66.82	90.56
2	2	2	1	230.00	0.93502	215.056	-2.40				
3	3	3	1	230.00	0.91903	211.376	-3.14	110.00	50.00		
4	4	4	1	230.00	0.92832	213.513	-2.38	100.00	70.00		
5	5	5	1	230.00	0.92562	212.893	-2.28	100.00	65.00		
6	6	6	1	230.00	0.93894	215.957	-1.22				
7	7	7	1	18.00	0.99999	18.000	6.49	-50.00	-16.43	200.00	95.62

Figure 49: PowerWorld implementation of Scenario 1

Comparing the simulation results with the PowerWorld results, we can see that the simulation correctly interprets the role of the `Solar` object added to the circuit. A few important notes, to replicate the behavior of a static PV generator in PowerWorld, a load with negative MW and MVAR is used. Furthermore, there is a small amount of error between the results in PowerWorld and the simulator. This is caused by the difference in how the `Solar` object is defined. In the simulator, the `Solar` object is defined with some real power as well as a power factor. Meanwhile, in PowerWorld the behavior of a PV generator requires a real power and reactive power. Since reactive power is internally calculated from power factor in the simulation and manually input by the user in PowerWorld, there is a small difference in the results. Finally, in the simulation, the power contribution from a `Solar` object counts as Gen MW, whereas in PowerWorld it falls under Load MW. However, this is unimportant regarding the internal calculations and the way the power flow is interpreted, and the result is the same despite this conflict.

Scenario II: Time-Varying Seven Bus System

The second important refactor of Validations.py was `Testing_Scenario2`. This scenario is similar to the first with a few changes. First, the `Solar` object is defined differently with 20 MW and unity power factor. Importantly, instead of performing a simple Newton-Raphson algorithm to validate this system, multiple iterations of the algorithm are performed with the `sweep_solar` method in the `Circuit` class. As noted in the documentation for this refactor, this method takes in two booleans: The first is for including time-varying load analysis, and the second is for writing results to a csv. In this case, we want to analyze the Duck Curve and the data associated with it, so both of these parameters are set to true.

```
def Testing_Scenario2():
    # Scenario 2: Testing solar generation over time on seven bus
    system at bus7
    circ = CreateSevenPowerBusSystem()

    # Added PV generation at bus7 with 20 MW at 1 unity power
    factor
    circ.add_solar("Solar_Scenario2", "bus7", 20, 1, "unity")

    # Perform sweep over day on circuit (also output results to csv
     for analysis)
    print(f"PROJECT 3 SCENARIO 2: TEST\n")
    circ.sweep_solar(True, True)
```

Listing 2: Python function for Scenario 2 test setup.

Ultimately, this generates a csv file containing the results of 24 iterations of the Newton-Raphson algorithm. Graphing each of these iterations as a function of time, we see the Duck Curve behavior emerge.

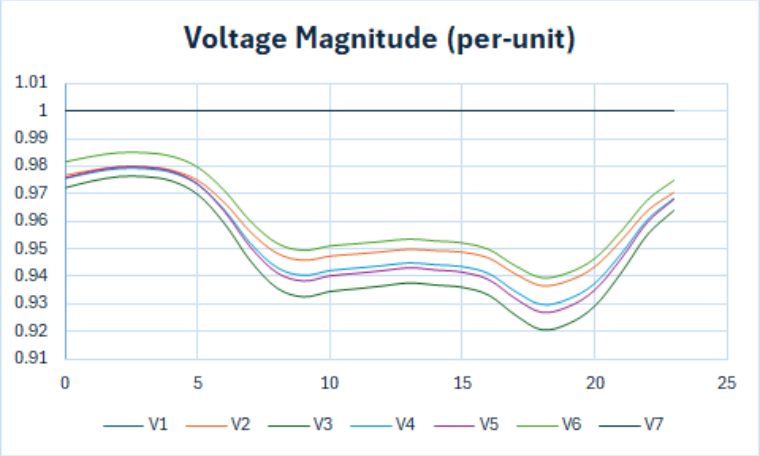


Figure 50: Voltage magnitude results from simulation [pu]

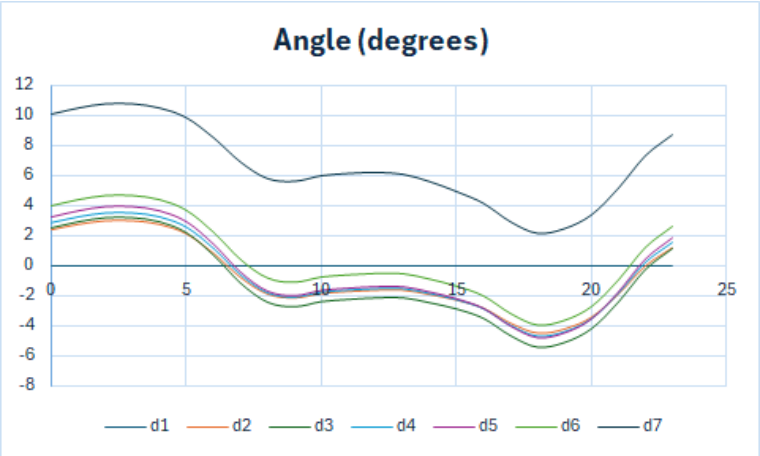


Figure 51: Angle results from simulation [degrees]

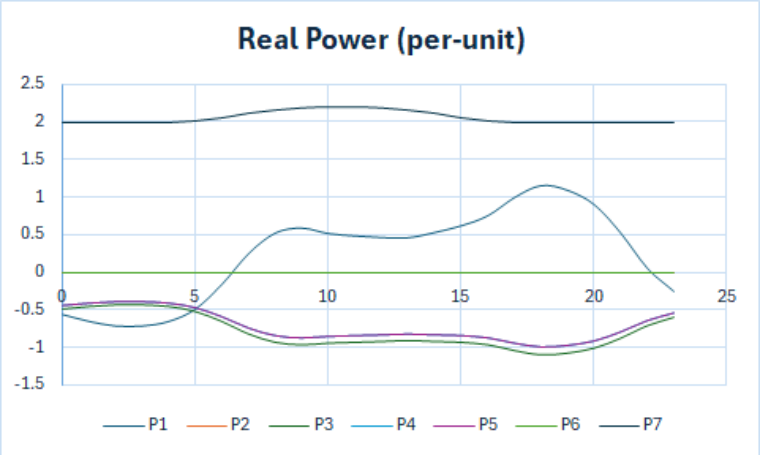


Figure 52: Real power results from simulation [pu]

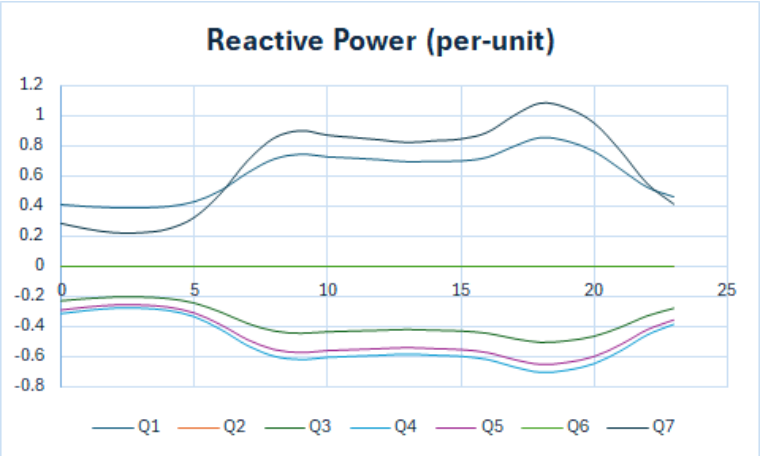


Figure 53: Reactive power results from simulation [pu]

Notably, observing the behavior in the Real Power graph, we see that the real power at bus1 follows the exact expected shape of the Duck Curve. Meanwhile, at bus7 we see real power held constant until irradiance from the sun provides a boost to the system. To validate these results, all of these iterations could be tested in PowerWorld. For the sake of brevity, only three examples will be explored: The results at the first, twelfth, and nineteenth hours. Note that the twelfth hour is when the PV generation reaches its maximum and the nineteenth hour is the hour at which the Gen MW at bus1 reaches its maximum.

Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	-56.95	41.12
2	bus2	230.0	0.97691	224.689	2.424	0.0	0.0	0.0	0.0
3	bus3	230.0	0.97242	223.656	2.554	49.5	22.5	0.0	0.0
4	bus4	230.0	0.97579	224.432	2.856	45.0	31.5	0.0	0.0
5	bus5	230.0	0.97599	224.478	3.268	45.0	29.25	0.0	0.0
6	bus6	230.0	0.98169	225.788	4.003	0.0	0.0	0.0	0.0
7	bus7	18.0	1.0	18.0	10.047	0.0	0.0	199.99	28.77

Table 6: Newton-Raphson Power Flow Results for Scenario 2 after first hour

Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	48.4	72.06
2	bus2	230.0	0.94838	218.127	-1.684	0.0	0.0	0.0	0.0
3	bus3	230.0	0.93547	215.159	-2.256	93.5	42.5	0.0	0.0
4	bus4	230.0	0.94307	216.906	-1.633	85.0	59.5	0.0	0.0
5	bus5	230.0	0.94102	216.435	-1.512	85.0	55.25	0.0	0.0
6	bus6	230.0	0.952	218.959	-0.603	0.0	0.0	0.0	0.0
7	bus7	18.0	1.0	18.0	6.114	0.0	0.0	220.0	85.91

Table 7: Newton-Raphson Power Flow Results for Scenario 2 after twelfth hour

Number	Name	Nom kV	PU Volt	Volt (kV)	Angle(Deg)	Load MW	Load MVAR	Gen MW	Gen MVAR
1	bus1	20.0	1.0	20.0	0.0	0.0	0.0	115.9	85.8
2	bus2	230.0	0.93692	215.492	-4.445	0.0	0.0	0.0	0.0
3	bus3	230.0	0.92049	211.712	-5.466	110.0	50.0	0.0	0.0
4	bus4	230.0	0.9298	213.854	-4.704	100.0	70.0	0.0	0.0
5	bus5	230.0	0.92673	213.147	-4.835	100.0	65.0	0.0	0.0
6	bus6	230.0	0.93968	216.127	-3.953	0.0	0.0	0.0	0.0
7	bus7	18.0	1.0	18.0	2.149	0.0	0.0	200.0	108.8

Table 8: Newton-Raphson Power Flow Results for Scenario 2 after nineteenth hour

To confirm these results with PowerWorld, the loads must be changed dynamically as well as the solar generation depending on the time. Iterating through each of these hours, we eventually see that the results match.

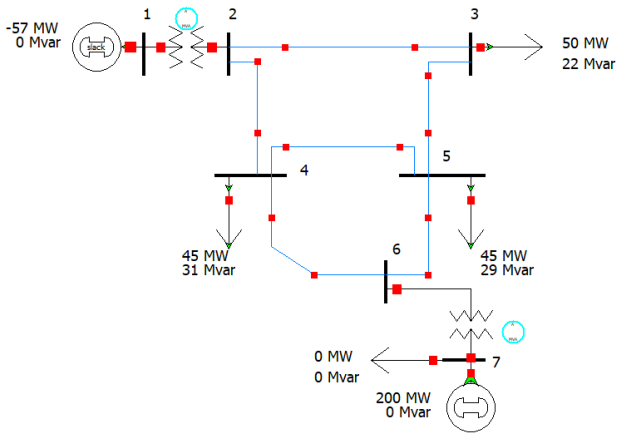


Figure 54: PowerWorld Diagram for Hour 1

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			-56.94	41.16
2	2	2	1	230.00	0.97688	224.681	2.42				
3	3	3	1	230.00	0.97238	223.648	2.55	49.50	22.50		
4	4	4	1	230.00	0.97576	224.424	2.86	45.00	31.50		
5	5	5	1	230.00	0.97595	224.469	3.27	45.00	29.25		
6	6	6	1	230.00	0.98165	225.780	4.00				
7	7	7	1	18.00	1.00000	18.000	10.05	0.00	0.00	200.00	28.84

Figure 55: PowerWorld Results for Hour 1

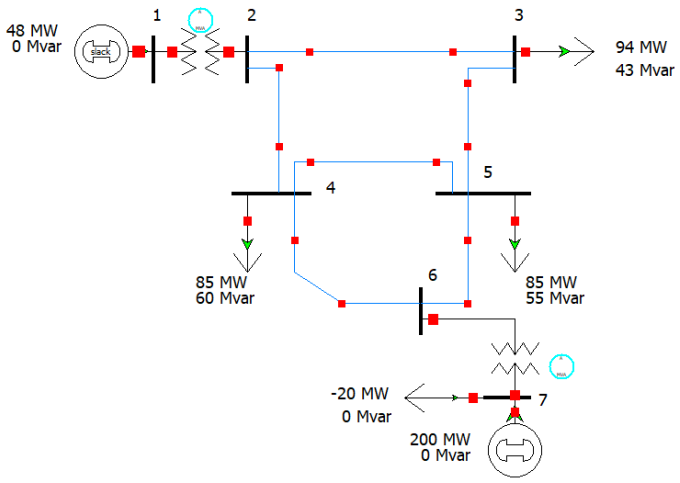


Figure 56: PowerWorld Diagram for Hour 12

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			48.39	72.01
2	2	2	1	230.00	0.94841	218.134	-1.68				
3	3	3	1	230.00	0.93551	215.168	-2.26	93.50	42.50		
4	4	4	1	230.00	0.94310	216.913	-1.63	85.00	59.50		
5	5	5	1	230.00	0.94105	216.443	-1.51	85.00	55.25		
6	6	6	1	230.00	0.95202	218.965	-0.60				
7	7	7	1	18.00	1.00000	18.000	6.11	-20.00	0.00	200.00	85.85

Figure 57: PowerWorld Results for Hour 12

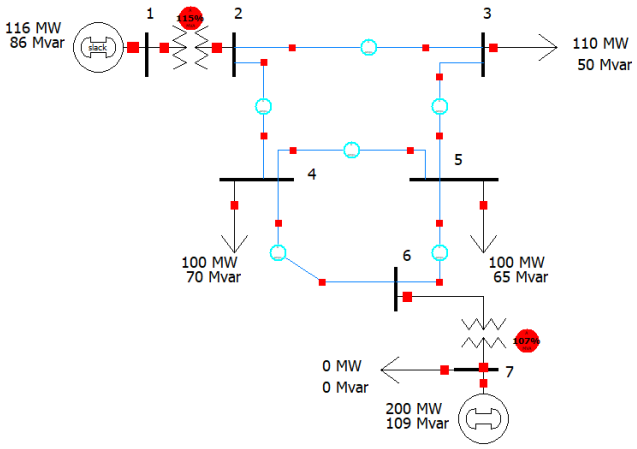


Figure 58: PowerWorld Diagram for Hour 19

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Load MW	Load Mvar	Gen MW	Gen Mvar
1	1	1	1	20.00	1.00000	20.000	0.00			115.90	85.80
2	2	2	1	230.00	0.93692	215.492	-4.45				
3	3	3	1	230.00	0.92049	211.712	-5.47	110.00	50.00		
4	4	4	1	230.00	0.92980	213.854	-4.70	100.00	70.00		
5	5	5	1	230.00	0.92673	213.147	-4.84	100.00	65.00		
6	6	6	1	230.00	0.93968	216.127	-3.95				
7	7	7	1	18.00	1.00000	18.000	2.15	0.00	0.00	200.00	108.80

Figure 59: PowerWorld Results for Hour 19

Notably, at the nineteenth hour the load happens to be at its maximum while the contribution from solar generation is zero. In this special case the results actually exactly match what we would expect from the system in its default state without any PV generation.